Lab Assignment 7 — Multiobjective Optimization: 37 Algorithms, Trade-offs and Applications

Optimization Techniques Lab · Yash Deshmukh · Roll 241070018 (R = 18)

Everything below is written from scratch in **Python + NumPy + Matplotlib only** (Section 2A). Run top to bottom. Single notebook layout:

- **A1** `moo_utils` (dominance, Deb rules, sorting, crowding, HV, IGD, Spacing) + problems P1–P14 + unit tests
- **A2/A3** all 37 algorithms: docstring, unit test, fixed-seed runs, plots, HV / IGD / Spacing, implementation log
- **Problem studies** (the "Investigate" questions of P1–P14)
- **B** comparison table · **C** deep dive (algorithms **18 MOEA/D, 30 MOMA, 5 Value Function**) · **D** head-to-head · **E** selection · **F** viva

Conventions: every objective is **minimised** (Max f ≡ Min −f). Budget **10,000 evaluations**, seeds fixed (`seed=0` for single runs, seeds 0–9 for comparisons). Set `QUICK = True` below to smoke-test the notebook in ~2 minutes with smaller budgets.

A1 · Shared toolkit `moo_utils`

Dominance test, Deb's constrained dominance (3 rules), fast non-dominated sorting, crowding distance, 2-D hypervolume (exact) / Monte-Carlo hypervolume (M ≥ 3), IGD, Spacing, normalisation, Das–Dennis reference points.

Unit tests of `moo_utils` on hand-checkable cases

Car example: six cars (price in $k, fuel use in L/100 km, both minimised). *Deb's table*: four candidate designs (two feasible, two infeasible).

```
Car example, front 1: ['A', 'B', 'C', 'F'] | front 2: ['D', 'E']
```

a	b	winner	Deb rule
p1	p2	p1	3 (dominance)
p1	p3	p1	1 (feasible > infeasible)
p1	p4	p1	1 (feasible > infeasible)
p2	p3	p2	1 (feasible > infeasible)
p2	p4	p2	1 (feasible > infeasible)
p3	p4	p3	2 (smaller violation)

```
moo_utils unit tests: PASS
```

A1 · Problems P1–P14

Each problem returns objective values **F** (minimisation form) and a constraint violation **CV** (0 = feasible). `prob.norm(F)` maps to ≈[0,1] with the problem's ideal/nadir; the HV reference point is **1.1** in normalised space for every problem. Reference (true) fronts `prob.Z` : analytic for P1, P2, P3, P7, P10, P14; dense sampling + Pareto filter for P4, P5, P6 (labelled *reference*); P8/P9 have none (best-known union is used when needed).

```
Problems ready: P1, P2, P3, P4, P5, P6, P7, P8, P9, P10_3, P10_5, P11, P14, P2i
```

Sanity checks from the problem bank (all verified by hand in the statement)

```
P1–P14 sanity checks: PASS | P8 coordinates (seed 7):
 [[7.63, 77.99], [43.84, 72.35], [97.8, 53.85], [50.11, 7.21], [26.84, 49.99], [67.92,
80.37], [38.09, 6.59], [28.81, 90.96], [21.34, 45.21], [93.12, 2.49], [60.05, 95.01],
[23.03, 54.85], [90.91, 13.32], [52.34, 75.04], [66.9, 46.78]]
P8 shortest-distance tour objs: [351.8 463.7] | shortest-time tour objs: [411.4 366.3]
```

A2 · Shared run / plot / log helpers

A2 · Classical methods (algorithms 1–9)

Sub-problem solver. Section 2A allows `scipy.optimize` for the single-objective sub-problem; to stay dependency-free this notebook uses its own constrained differential evolution (Deb feasibility rule) + pattern-search polish, `solve_so` . The scalarisation, sweeps over weights / ε / targets, deviation variables and priority logic are all coded below.

Unit tests (algorithms 1–9): each checks the key rule of the method

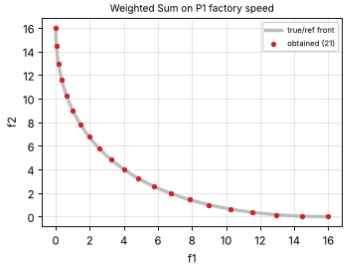
```
{1: 'PASS', 2: 'PASS', 3: 'PASS', 4: 'PASS', 5: 'PASS', 6: 'PASS', 7: 'PASS', 8: 'PASS',
9: 'PASS'}
```

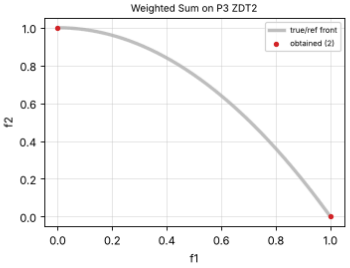
Alg 1 — Weighted Sum · Alg 2 — ε-constraint (runs on their test problems, seed 0)

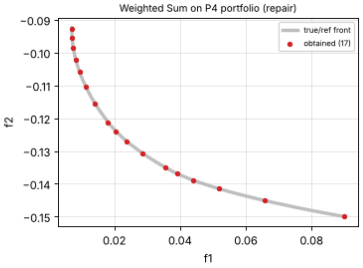
```
[ 1] Weighted Sum   P1      n=21   HV=1.0258  IGD=0.0202  Spacing=0.0089  nfe=172116
t=0.4s

[ 1] Weighted Sum   P3      n=2    HV=0.2100  IGD=0.3543  Spacing=nan  nfe=200000  t=0.6s

[ 1] Weighted Sum   P4      n=17   HV=0.9395  IGD=0.0344  Spacing=0.0625  nfe=180968
t=0.6s
```



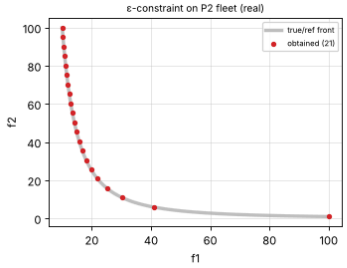


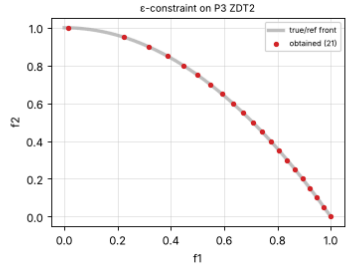


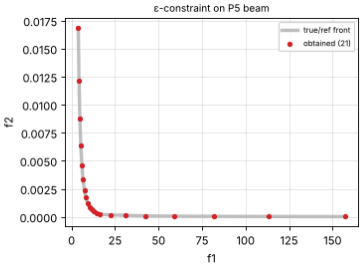

```
[ 2] ε-constraint   P2      n=21   HV=1.0865  IGD=0.0372  Spacing=0.1291  nfe=172148
t=0.4s

[ 2] ε-constraint   P3      n=21   HV=0.5192  IGD=0.0253  Spacing=0.0343  nfe=217200
t=0.8s

[ 2] ε-constraint   P5      n=21   HV=0.8188  IGD=0.0212  Spacing=0.0203  nfe=174592
t=0.9s
```



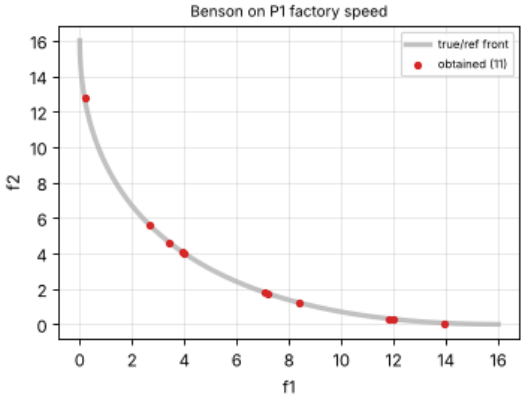


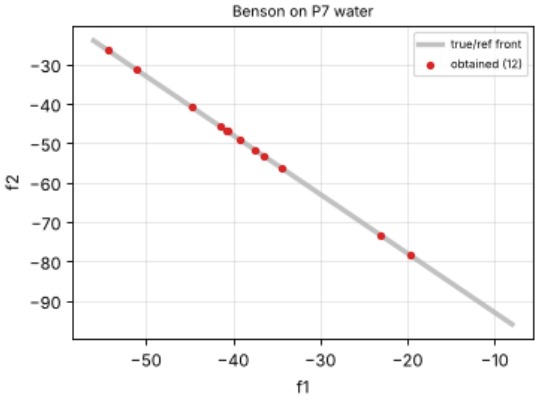


Alg 3 — Benson · Alg 4 — Weight metric (p = 1, 2, ∞)

```
[ 3] Benson          P1       n=11   HV=0.9703  IGD=0.0730  Spacing=0.1305  nfe=98352
t=0.3s

[ 3] Benson          P7       n=12   HV=0.6056  IGD=0.0759  Spacing=0.0375  nfe=100512
t=0.4s
```






```
[ 4] Wmetric p=1     P1_p1    n=21   HV=1.0265  IGD=0.0194  Spacing=0.0080  nfe=172116
t=0.4s

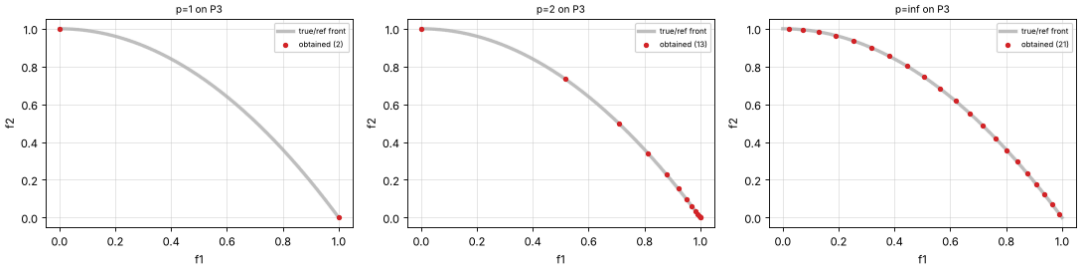
[ 4] Wmetric p=1     P3_p1    n=2    HV=0.2100  IGD=0.3543  Spacing=nan   nfe=200360  t=0.7s

[ 4] Wmetric p=2     P1_p2    n=21   HV=1.0268  IGD=0.0261  Spacing=0.0287  nfe=172116
t=0.4s

[ 4] Wmetric p=2     P3_p2    n=13   HV=0.4616  IGD=0.0998  Spacing=0.1593  nfe=203000
t=0.8s

[ 4] Wmetric p=inf   P1_pinf  n=21   HV=1.0217  IGD=0.0433  Spacing=0.0335  nfe=172116
t=0.4s

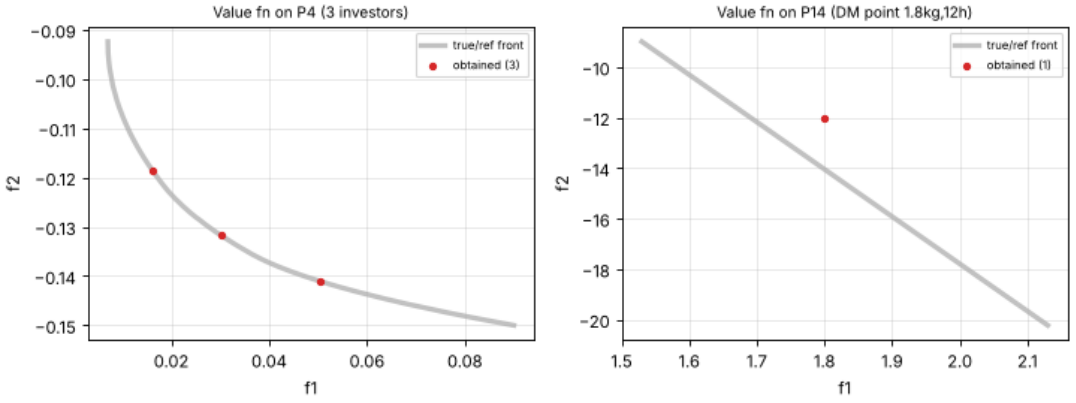
[ 4] Wmetric p=inf   P3_pinf  n=21   HV=0.5168  IGD=0.0180  Spacing=0.0108  nfe=218080
t=0.9s
```



Alg 5 — Value function (P4 portfolio and P14 laptop) · Alg 6 — Goal programming · Alg 7 — Lexicographic

investor	weight a on risk	weights w	risk	return	utility
conservative	0.8	[0.061, 0.34, 0.26, 0.338]	0.01616	0.1186	0.7783
balanced	0.5	[0.0, 0.452, 0.454, 0.094]	0.03012	0.1317	0.7021
aggressive	0.2	[0.0, 0.298, 0.702, 0.0]	0.05041	0.1411	0.7536

```
P14 value-function solution: screen 14.53 in, battery 56.1 Wh -> 1.800 kg, 12.00 h
```



Goal programming, P11 (goals: profit ≥ 2400, units ≥ 600). Unit weights vs weights normalised by the goal values:

weights	(xA,xB)	(profit,units)	d-	d+	objective
unit weights	[300.0, 200.0]	[2400.0, 500.0]	[0.0, 100.0]	[0.0, 0.0]	100
normalised (1/goal)	[300.0, 200.0]	[2400.0, 500.0]	[0.0, 100.0]	[0.0, 0.0]	0.1667
priority-like (1,100)	[300.0, 200.0]	[2400.0, 500.0]	[0.0, 100.0]	[0.0, 0.0]	1e+04

Lexicographic, P11 (profit first, units second) and **why the tie must be broken**: profit = 2400 on the whole edge from (300,200) to (0,400), units are 500 vs 400 there, so only the 2nd priority picks (300,200).

priority	(xA,xB)	(profit,units)
profit only	[0.0, 400.0]	[2400.0, 400.0]
profit then units	[300.0, 200.0]	[2400.0, 500.0]

P7 water allocation (goals: yield ≥ 40, output ≥ 50; reference point of the DM = (40, 50)). Both goals together would need 91.7 > 90 million m^3, so a shortfall is unavoidable:

weights	(x1,x2)	(yield,output)	d-
unit weights	[48.33, 41.67]	[38.67, 50.0]	[1.33, 0.0]
normalised	[48.33, 41.67]	[38.67, 50.0]	[1.33, 0.0]
agri-first (10,1)	[50.0, 40.0]	[40.0, 48.0]	[0.0, 2.0]

priority	(x1,x2)	(yield,output)
agri first	[70.0, 20.0]	[56.0, 24.0]
industry first	[10.0, 80.0]	[8.0, 96.0]

Alg 8 — TOPSIS · Alg 9 — VIKOR on P12 (supplier selection)

```
Pareto-optimal suppliers: ['S1', 'S2', 'S3', 'S4', 'S5']
TOPSIS closeness: [0.6214 0.4758 0.6298 0.3702 0.597 ] ranking: ['S3', 'S1', 'S5', 'S2',
'S4']
```

v	Q	ranking	C1	C2
0.25	[0.047, 0.238, 0.75, 1.0, 0.464]	['S1', 'S2', 'S5', 'S3', 'S4']	False	True
0.5	[0.093, 0.309, 0.5, 1.0, 0.429]	['S1', 'S2', 'S5', 'S3', 'S4']	False	True
0.75	[0.14, 0.38, 0.25, 1.0, 0.393]	['S1', 'S3', 'S2', 'S5', 'S4']	False	True

weight shift	TOPSIS ranking	VIKOR (v=.5) ranking
+0.1 cost / -0.1 quality	['S1', 'S2', 'S3', 'S5', 'S4']	['S2', 'S1', 'S3', 'S5', 'S4']
-0.1 cost / +0.1 quality	['S3', 'S5', 'S1', 'S2', 'S4']	['S1', 'S3', 'S5', 'S2', 'S4']
+0.1 delivery / -0.1 reliability	['S3', 'S5', 'S1', 'S2', 'S4']	['S1', 'S2', 'S5', 'S3', 'S4']

```
With bad-criterion supplier S6 (reliability 40%): TOPSIS rank of S6 = 1 | VIKOR v=0 (max
regret) rank of S6 = 3 | VIKOR v=.5 rank = 1
```

A2 · Evolutionary framework (shared by algorithms 10–35)

`Variation` (SBX + polynomial mutation for real, order crossover + swap/inversion for permutations, uniform crossover + bit flip for binary), base class `MOO` with `initialize / evaluate / iterate / run`, an external-archive helper `Arc`, selection helpers and the algorithm registry. Algorithms **without** feasibility rules handle constraints with a static penalty (`prob.pen`); only Constrained NSGA-II (21) uses Deb's rules.

Family: non-elitist evolutionary methods (algorithms 10–13)

Family: elitist methods — NSGA-II (14), Constrained NSGA-II (21), MOMA (30), R-NSGA-II (35)

Family: reference-point, archive-based and decomposition methods (algorithms 15–20)

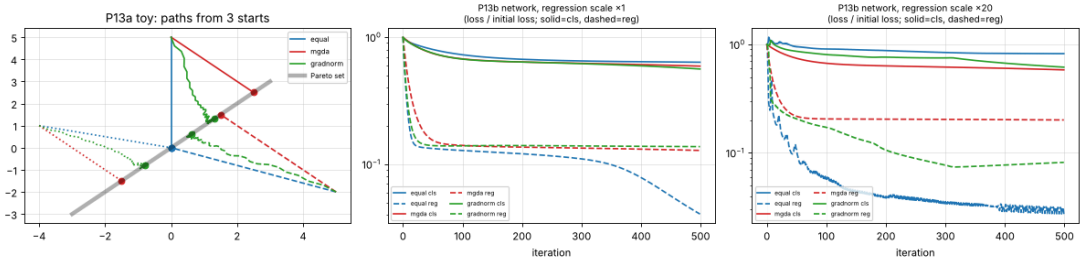
Family: swarm, physics- and human-inspired metaheuristics (algorithms 22–29)

Family: many-objective and indicator-based methods (algorithms 31–34)

Family: gradient-based multi-task methods (algorithms 36–37) on P13

(a) Toy: f1 = ‖x−a‖², f2 = ‖x−b‖², a=(3,3), b=(−3,−3), start (0,5); Pareto set = segment a–b. **(b) Network:** a shared body (tanh layer) with a classification head (logistic loss) and a regression head (MSE), forward/backward written by hand in NumPy.

Run: toy (a) and network (b), balanced and imbalanced (regression targets ×20 ⇒ much larger gradients)



reg scale	method	cls loss start	cls loss end	reg loss start	reg loss end	verdict
1	equal	0.6922	0.4398	4.103	0.1663	both fall
1	mgda	0.6922	0.4098	4.103	0.5269	both fall
1	gradnorm	0.6922	0.3889	4.103	0.5644	both fall
20	equal	0.6922	0.5701	1143	31.2	both fall
20	mgda	0.6922	0.4037	1143	230	both fall
20	gradnorm	0.6922	0.4274	1143	93.48	both fall

```
Toy end points (start (0,5)): {'equal': [0.0, 0.0], 'mgda': [2.5, 2.5], 'gradnorm':
[1.327, 1.327]} -> Pareto set is x1 = x2
```

A2 · Unit tests for algorithms 10–37 (each checks the key rule of the algorithm)

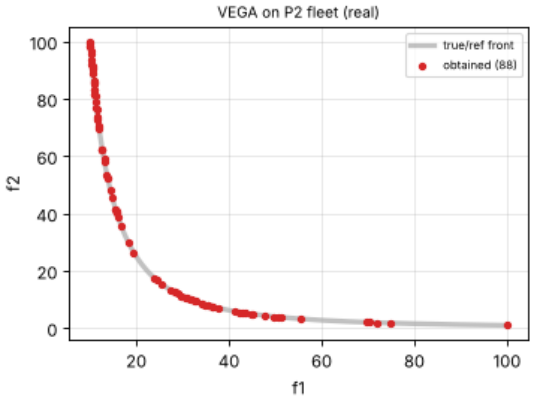
#	algorithm	unit test
1	Weighted Sum	PASS
2	ε-Constraint	PASS
3	Benson	PASS
4	Weight Metric	PASS
5	Value Function	PASS
6	Goal Programming	PASS
7	Lexicographic	PASS
8	TOPSIS	PASS
9	VIKOR	PASS
10	VEGA	PASS
11	MOGA	PASS
12	NSGA	PASS
13	NPGA	PASS
14	NSGA-II	PASS
15	NSGA-III	PASS
16	SPEA2	PASS
17	PESA-II	PASS
18	MOEA/D	PASS
19	MOEA/D-DE	PASS
20	MOEA/D-ACO	PASS
21	Constrained NSGA-II	PASS
22	MOPSO	PASS
23	MOGWO	PASS
24	MOABC	PASS
25	MOACO	PASS
26	MOSA	PASS
27	MODE	PASS
28	MO-TLBO	PASS
29	MOBA	PASS
30	MOMA	PASS
31	RVEA	PASS
32	IBEA	PASS
33	SMS-EMOA	PASS
34	HypE	PASS

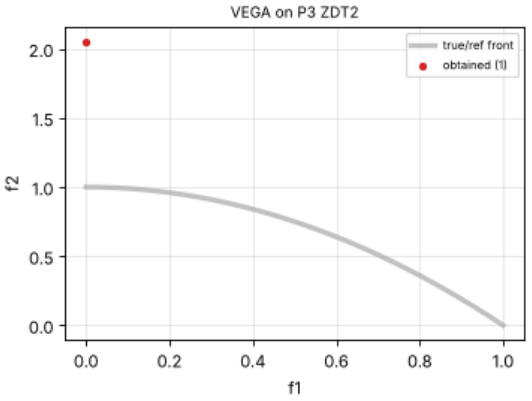
#	algorithm	unit test
35	R-NSGA-II	PASS
36	MGDA	PASS
37	GradNorm	PASS

A2 · Runs with a fixed seed (seed 0, 10,000 evaluations; P9 uses 3,000 because every evaluation is a 5-fold 5-NN cross-validation)

Each algorithm is run on the problems listed for it in the catalogue. Plots show the obtained front (red) over the true / reference front (grey). Budget = number of function evaluations (the actual count can exceed 10,000 by at most one generation).

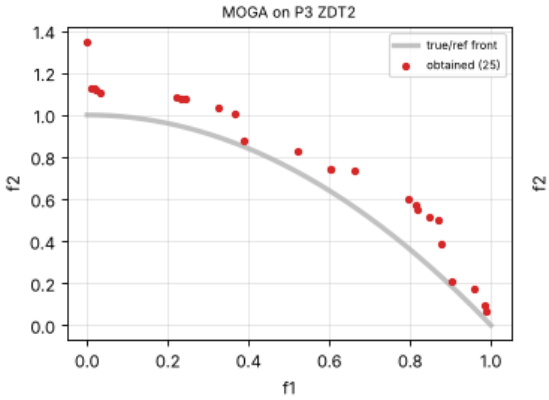
```
[10] VEGA           P2      n=88    HV=1.1112  IGD=0.0128  Spacing=0.0301  nfe=10000
t=0.0s
[10] VEGA           P3      n=1     HV=0.0000  IGD=1.4849  Spacing=nan  nfe=10000  t=0.0s
```

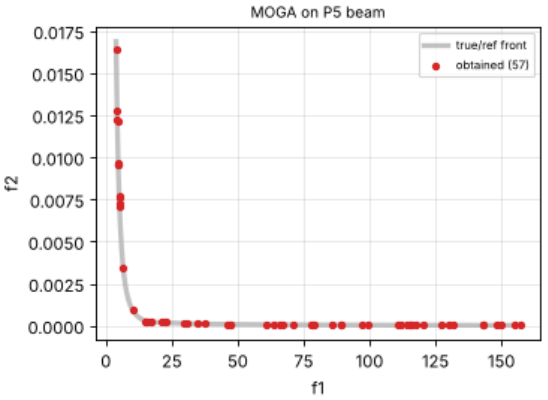




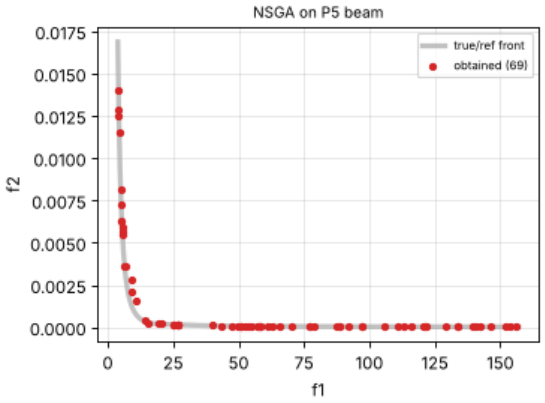

```
[11] MOGA           P3      n=25    HV=0.3862  IGD=0.0900  Spacing=0.0513  nfe=10000
t=0.2s

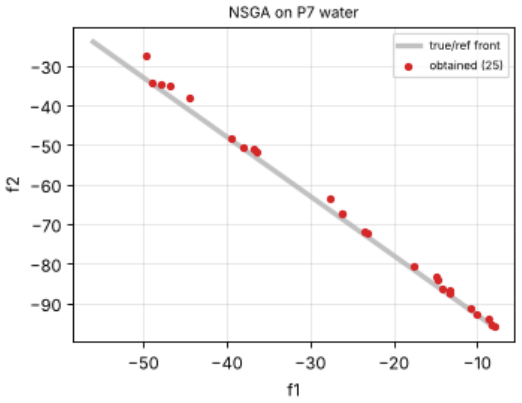
[11] MOGA           P5      n=57    HV=0.8083  IGD=0.0237  Spacing=0.0322  nfe=10000
t=0.2s
```





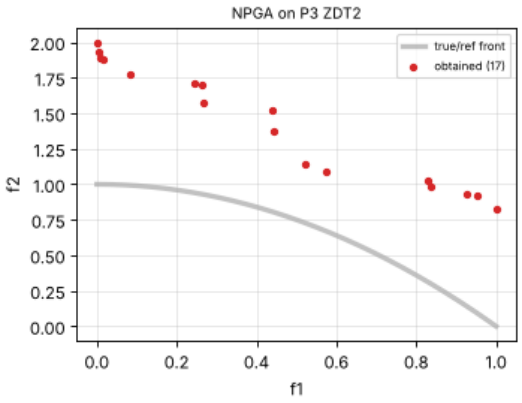

```
[12] NSGA           P5      n=69    HV=0.8023  IGD=0.0225  Spacing=0.0157  nfe=10000
t=0.2s
[12] NSGA           P7      n=25    HV=0.6318  IGD=0.0417  Spacing=0.0225  nfe=10000
t=0.2s
```

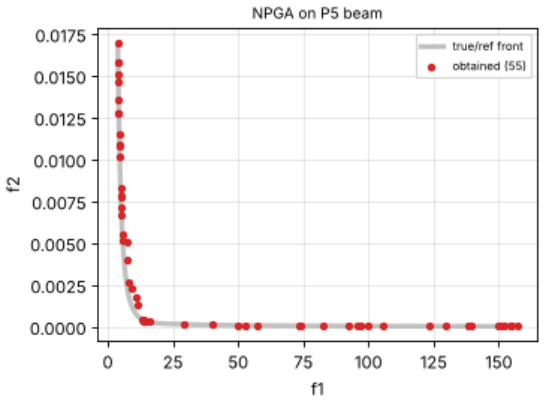





```
[13] NPGA           P3      n=17    HV=0.0549  IGD=0.4458  Spacing=0.0466  nfe=10000
t=0.4s

[13] NPGA           P5      n=55    HV=0.7965  IGD=0.0279  Spacing=0.0191  nfe=10000
t=0.5s
```


NPGA on P3 ZDT2
true/ref front
obtained (17)
f1
f2
0.0
0.2
0.4
0.6
0.8
1.0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

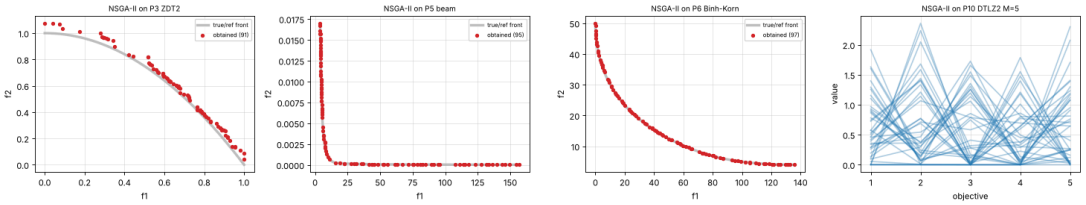

NPGA on P5 beam
true/ref front
obtained (55)
f1
f2
0
25
50
75
100
125
150
0.0000
0.0025
0.0050
0.0075
0.0100
0.0125
0.0150
0.0175

```
[14] NSGA-II        P3      n=91   HV=0.4869  IGD=0.0324  Spacing=0.0139  nfe=10000
t=0.4s

[14] NSGA-II        P5      n=95   HV=0.8278  IGD=0.0132  Spacing=0.0109  nfe=10000
t=0.4s

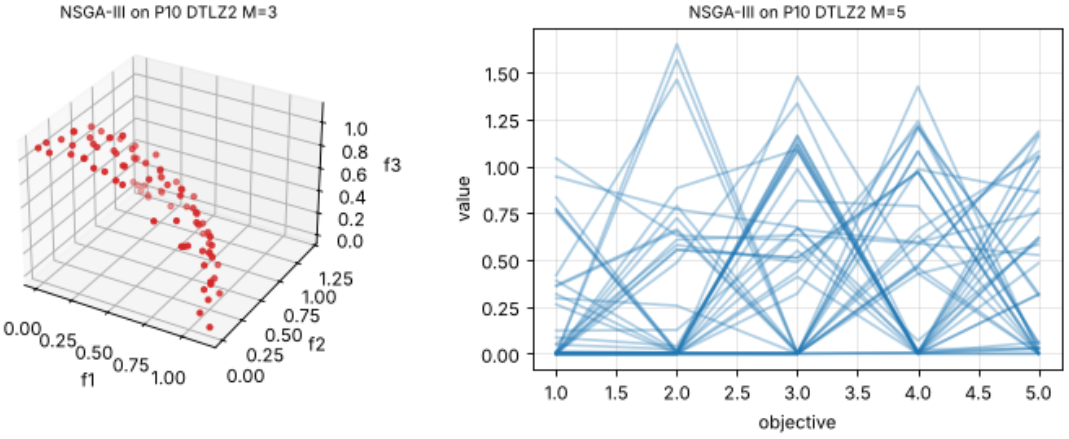
[14] NSGA-II        P6      n=97   HV=1.0196  IGD=0.0057  Spacing=0.0051  nfe=10000
t=0.4s

[14] NSGA-II        P10_5   n=100  HV=0.1314  IGD=0.6545  Spacing=0.1191  nfe=10000
t=0.5s
```



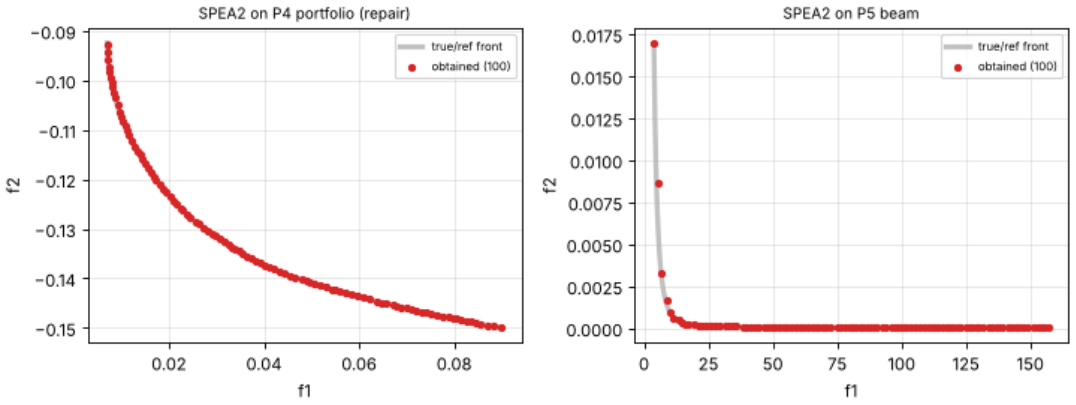

```
[15] NSGA-III       P10_3   n=92   HV=0.5899  IGD=0.1036  Spacing=0.0639  nfe=10028
t=0.6s

[15] NSGA-III       P10_5   n=72   HV=0.4879  IGD=0.4963  Spacing=0.1584  nfe=10008
t=0.6s
```



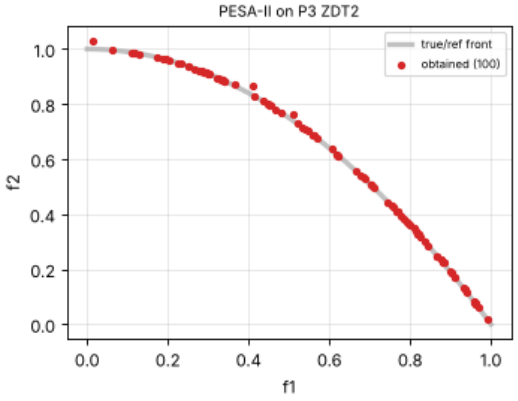

```
[16] SPEA2          P4      n=100  HV=0.9701  IGD=0.0039  Spacing=0.0034  nfe=10000
t=2.5s

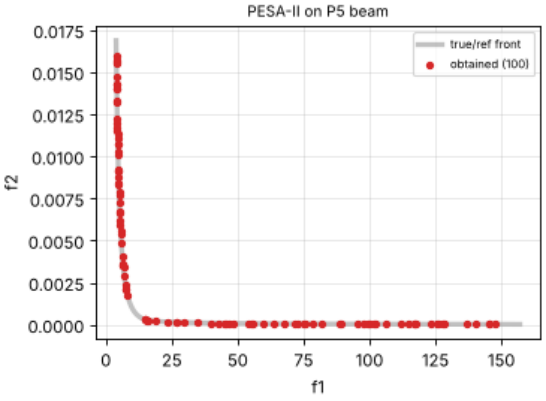
[16] SPEA2          P5      n=100  HV=0.8155  IGD=0.0127  Spacing=0.0250  nfe=10000
t=2.3s
```




```
[17] PESA-II        P3      n=100  HV=0.5287  IGD=0.0089  Spacing=0.0107  nfe=10000
t=3.2s
```

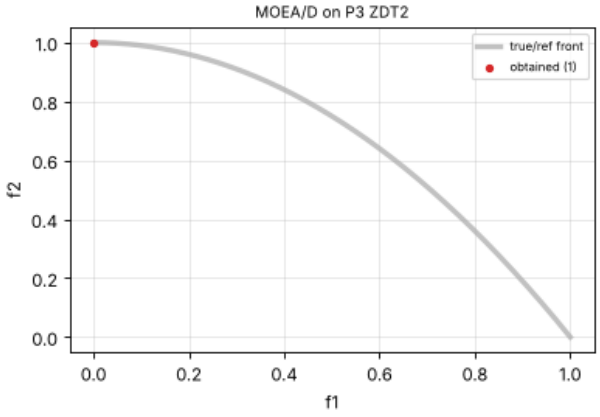
```
[17] PESA-II        P5      n=100  HV=0.8135  IGD=0.0215  Spacing=0.0098  nfe=10000
t=3.5s
```

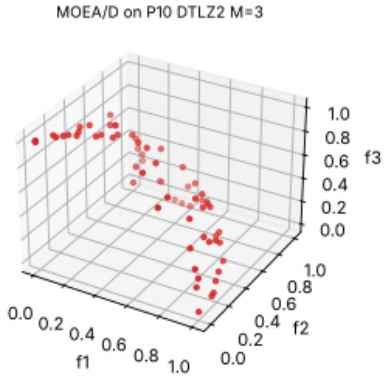




```
[18] MOEA/D          P3       n=1    HV=0.1100  IGD=0.6097  Spacing=nan   nfe=10000  t=3.5s

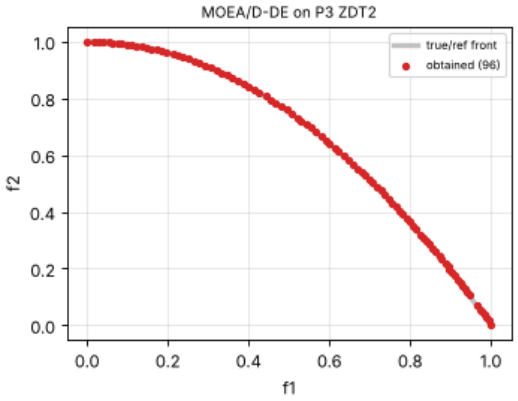
[18] MOEA/D          P10_3    n=70   HV=0.6786  IGD=0.0874  Spacing=0.0601  nfe=10080
t=5.0s
```

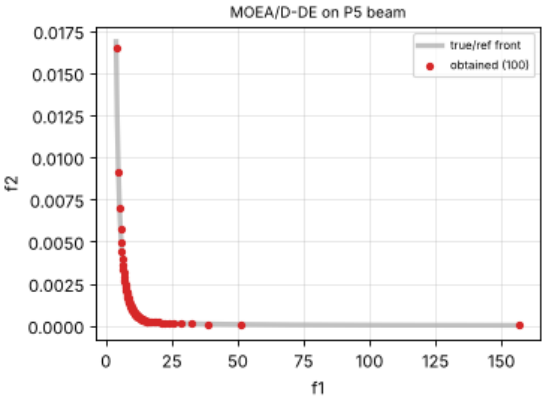




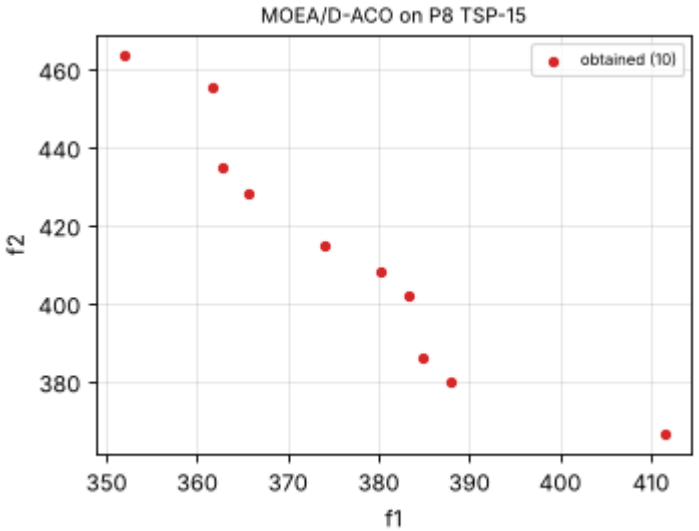

```
[19] MOEA/D-DE       P3       n=96   HV=0.5357  IGD=0.0045  Spacing=0.0033  nfe=10000
t=4.5s

[19] MOEA/D-DE       P5       n=100  HV=0.8099  IGD=0.0338  Spacing=0.0368  nfe=10000
t=4.9s
```





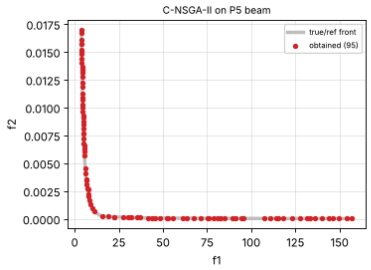

```
[20] MOEA/D-ACO      P8       n=10   HV=0.7566  IGD=nan   Spacing=0.1012  nfe=10000  t=3.9s
```

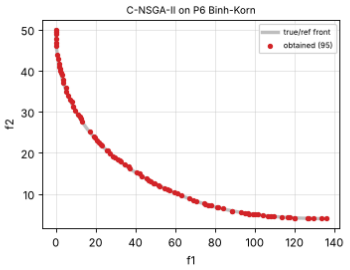


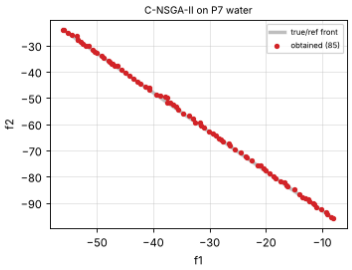

```
[21] C-NSGA-II       P5       n=95   HV=0.8278  IGD=0.0132  Spacing=0.0109  nfe=10000
t=0.4s
```

```
[21] C-NSGA-II      P6      n=95   HV=1.0189  IGD=0.0062  Spacing=0.0057  nfe=10000
t=0.5s

[21] C-NSGA-II      P7      n=85   HV=0.6992  IGD=0.0063  Spacing=0.0058  nfe=10000
t=0.5s
```

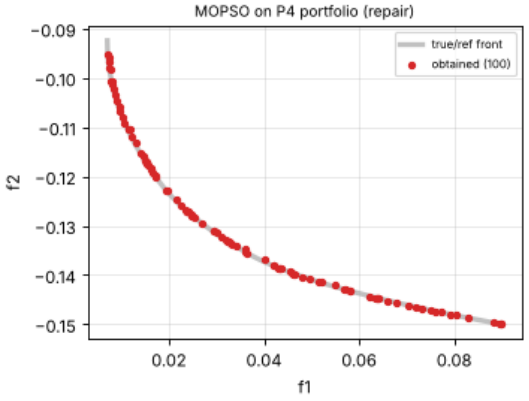


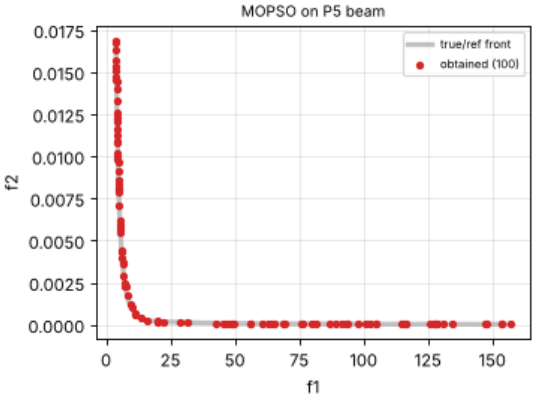




```
[22] MOPSO          P4      n=100  HV=0.9656  IGD=0.0073  Spacing=0.0082  nfe=10000
t=0.7s

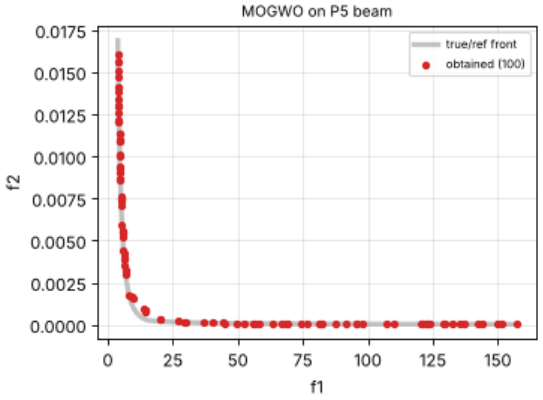
[22] MOPSO          P5      n=100  HV=0.8304  IGD=0.0107  Spacing=0.0131  nfe=10000
t=0.8s
```

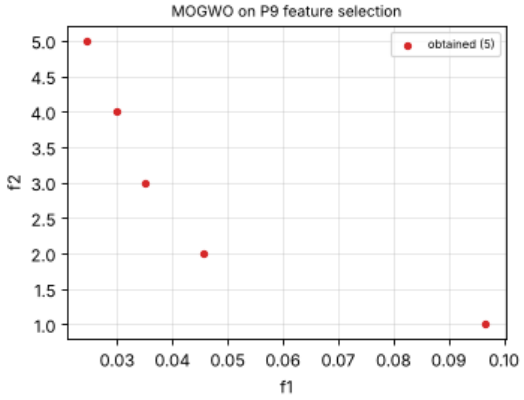





```
[23] MOGWO          P5      n=100  HV=0.7908  IGD=0.0311  Spacing=0.0088  nfe=10000
t=0.6s

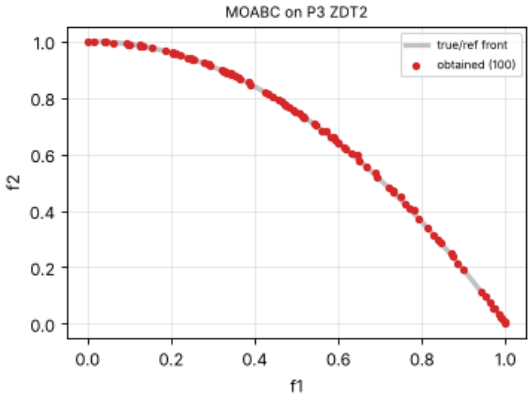
[23] MOGWO          P9      n=5    HV=1.1329  IGD=nan  Spacing=0.0375  nfe=3000  t=12.4s
```

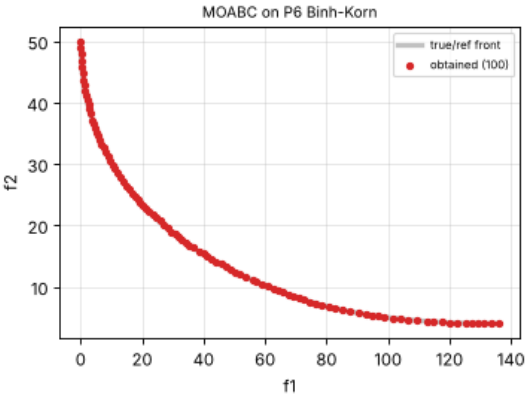




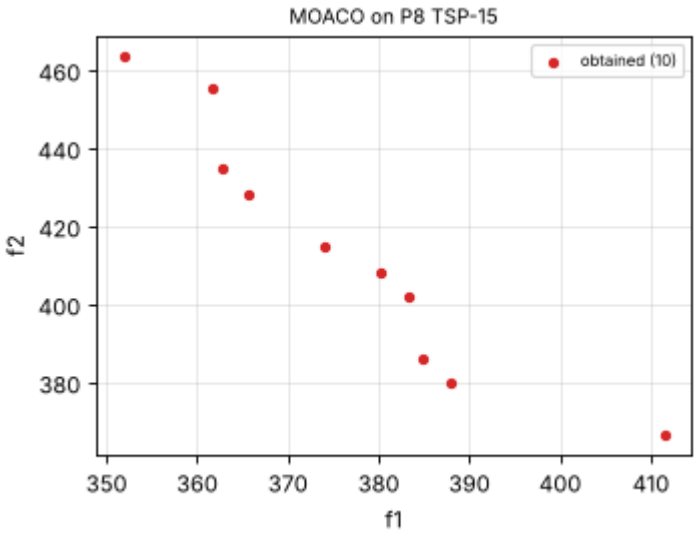

```
[24] MOABC          P3      n=100  HV=0.5341  IGD=0.0066  Spacing=0.0070  nfe=10050
t=0.4s

[24] MOABC          P6      n=100  HV=1.0211  IGD=0.0043  Spacing=0.0028  nfe=10050
t=0.6s
```



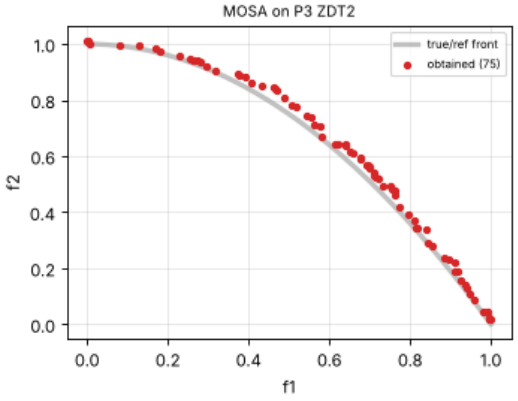


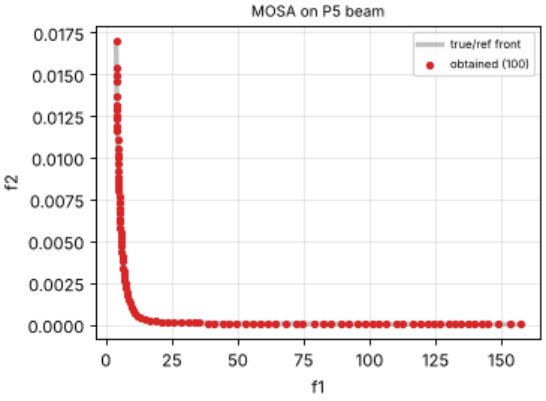

```
[25] MOACO          P8      n=10   HV=0.7566  IGD=nan  Spacing=0.1012  nfe=10000  t=1.9s
```


MOACO on P8 TSP-15
obtained (10)
460
440
420
400
380
f2
350
360
370
380
390
400
410
f1

[26] MOSA P3 n=75 HV=0.5108 IGD=0.0189 Spacing=0.0100 nfe=10000 t=0.8s

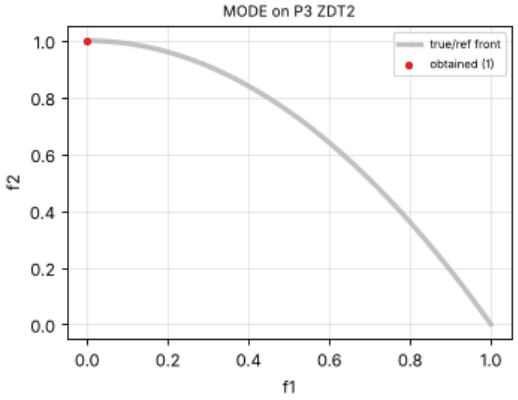
[26] MOSA P5 n=100 HV=0.8355 IGD=0.0060 Spacing=0.0104 nfe=10000 t=4.8s

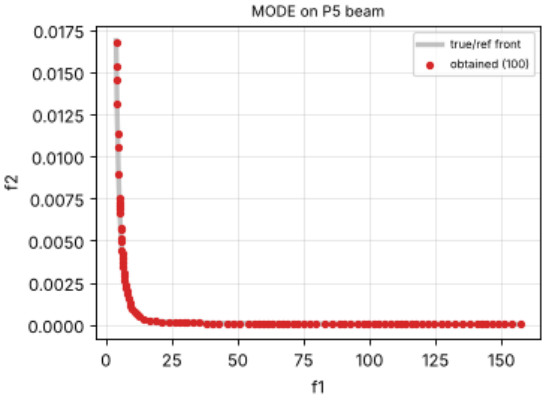




[27] MODE P3 n=1 HV=0.1097 IGD=0.6098 Spacing=nan nfe=10000 t=0.3s

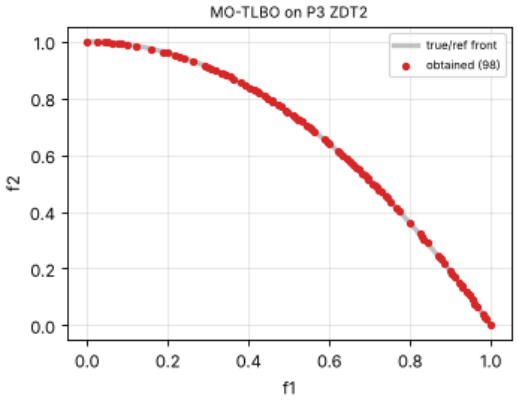
[27] MODE P5 n=100 HV=0.8358 IGD=0.0059 Spacing=0.0096 nfe=10000 t=0.7s

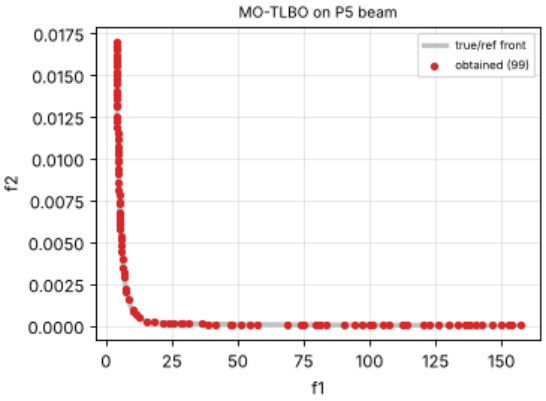




[28] MO-TLBO P3 n=98 HV=0.5367 IGD=0.0053 Spacing=0.0064 nfe=10100 t=0.4s

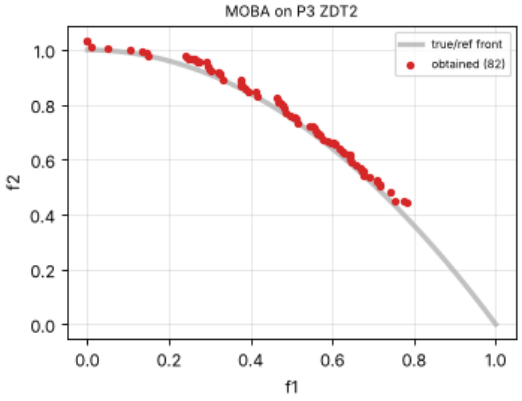
[28] MO-TLBO P5 n=99 HV=0.8305 IGD=0.0104 Spacing=0.0103 nfe=10100 t=0.4s

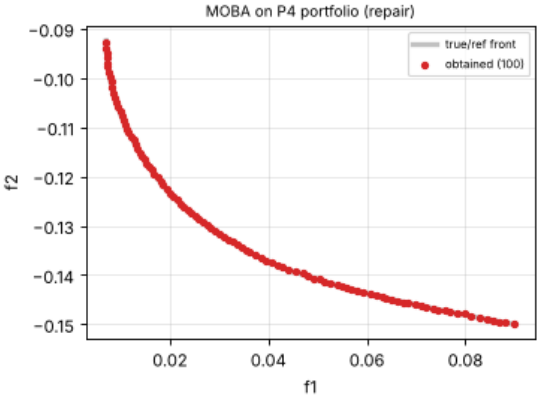




[29] MOBA P3 n=82 HV=0.4301 IGD=0.0710 Spacing=0.0073 nfe=10000 t=0.3s

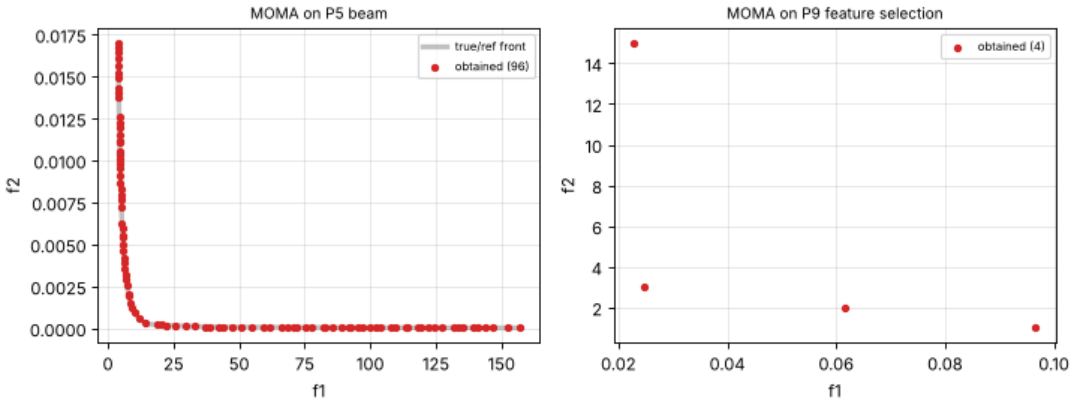
```
[29] MOBA           P4      n=100  HV=0.9698  IGD=0.0041  Spacing=0.0022  nfe=10000
t=0.6s
```



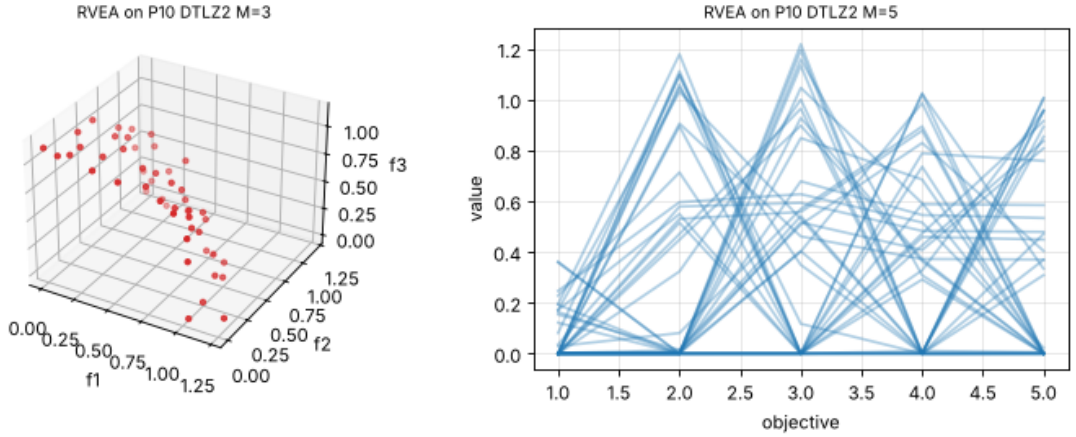


```
[30] MOMA            P5      n=96    HV=0.8303  IGD=0.0098  Spacing=0.0147  nfe=10035
t=0.2s

[30] MOMA            P9      n=4     HV=1.1357  IGD=nan  Spacing=0.1377  nfe=3045  t=11.1s
```

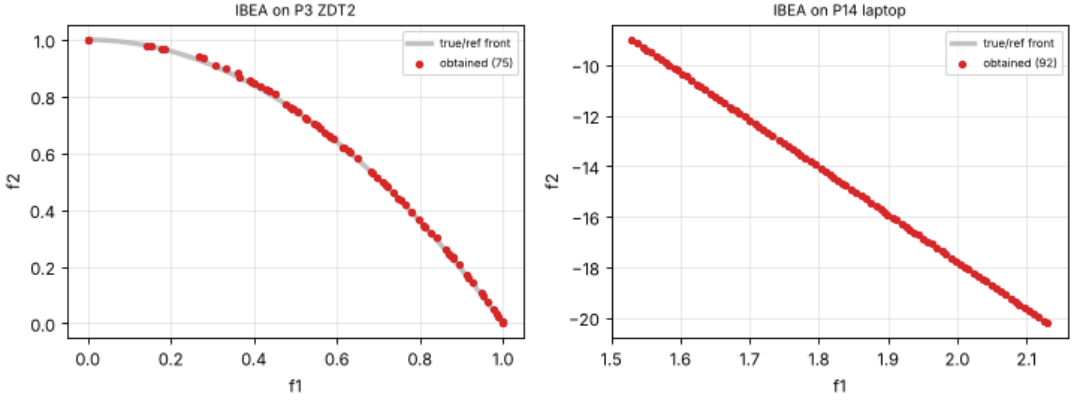



```
[31] RVEA            P10_3   n=50    HV=0.5133  IGD=0.1390  Spacing=0.0294  nfe=10010
t=0.1s
[31] RVEA            P10_5   n=40    HV=0.7942  IGD=0.4274  Spacing=0.1096  nfe=10010
t=0.1s
```



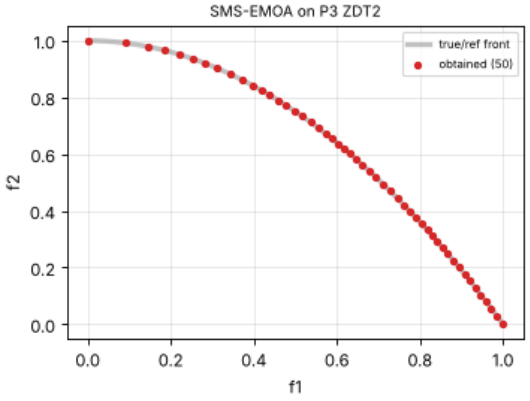

```
[32] IBEA            P3      n=75    HV=0.5303  IGD=0.0127  Spacing=0.0073  nfe=10000
t=0.5s

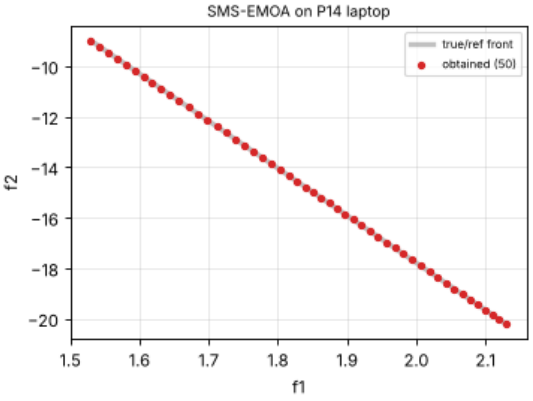
[32] IBEA            P14     n=92    HV=0.7043  IGD=0.0040  Spacing=0.0025  nfe=10000
t=0.4s
```




```
[33] SMS-EMOA        P3      n=50    HV=0.5343  IGD=0.0092  Spacing=0.0106  nfe=10000
t=4.4s
```

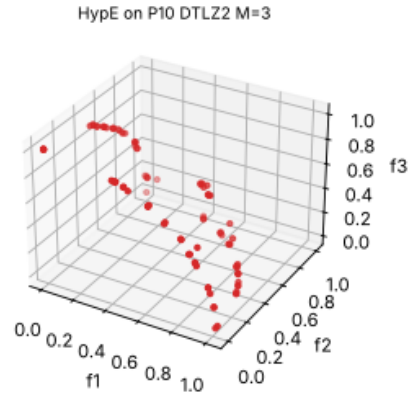
[33] SMS-EMOA P14 n=50 HV=0.6997 IGD=0.0072 Spacing=0.0021 nfe=10000 t=4.4s

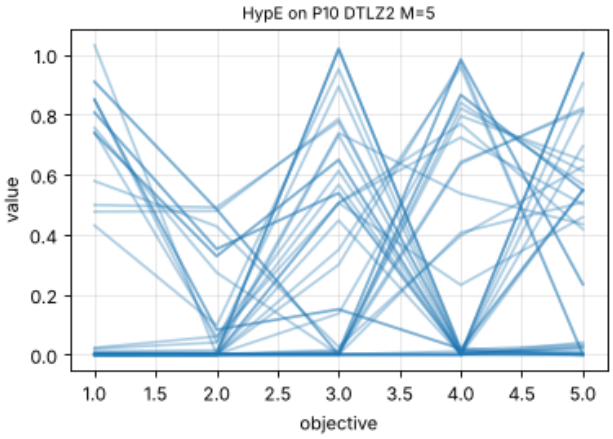




[34] HypE P10_3 n=100 HV=0.6946 IGD=0.1241 Spacing=0.0318 nfe=10000 t=3.3s

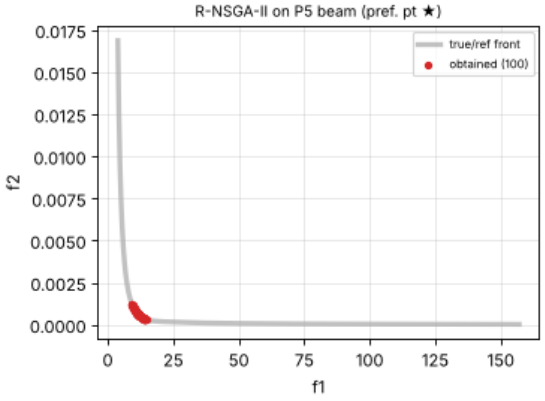
[34] HypE P10_5 n=100 HV=0.9898 IGD=0.3832 Spacing=0.0440 nfe=10000 t=5.6s

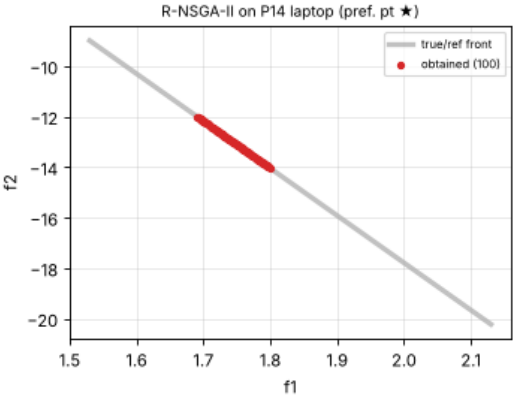




[35] R-NSGA-II P5 n=100 HV=0.6022 IGD=0.3092 Spacing=0.0003 nfe=10000 t=3.6s

[35] R-NSGA-II P14 n=100 HV=0.4394 IGD=0.2673 Spacing=0.0003 nfe=10000 t=3.6s





A3 · Implementation log

#	algorithm	problems	evaluations	HV	IGD	runtime	status
1	Weighted Sum	P1, P3, P4	172116/200000/180968	1.026 / 0.210 / 0.939	0.020 / 0.354 / 0.034	0.4s / 0.6s / 0.6s	works
2	ε-Constraint	P2, P3, P5	172148/217200/174592	1.086 / 0.519 / 0.819	0.037 / 0.025 / 0.021	0.4s / 0.8s / 0.9s	works
3	Benson	P1, P7	98352/100512	0.970 / 0.606	0.073 / 0.076	0.3s / 0.4s	works
4	Weight Metric	P1, P3	172116/218080	1.027 / 0.517	0.026 / 0.018	0.4s / 0.9s	works
5	Value Function	P4, P14	8376/8376	0.830 / 0.239	0.233 / 0.416	0.0s / 0.0s	works
6	Goal Programming	P11, P7	sweep/sweep	– / –	– / –	0.0s / 0.0s	works
7	Lexicographic	P11, P7	sweep/sweep	– / –	– / –	0.0s / 0.0s	works
8	TOPSIS	P12	sweep	–	–	0.0s	works
9	VIKOR	P12	sweep	–	–	0.0s	works
10	VEGA	P2, P3	10000/10000	1.111 / 0.000	0.013 / 1.485	0.0s / 0.0s	works
11	MOGA	P3, P5	10000/10000	0.386 / 0.808	0.090 / 0.024	0.2s / 0.2s	works
12	NSGA	P5, P7	10000/10000	0.802 / 0.632	0.022 / 0.042	0.2s / 0.2s	works
13	NPGA	P3, P5	10000/10000	0.055 / 0.797	0.446 / 0.028	0.4s / 0.5s	works
14	NSGA-II	P3, P5, P6, P10_5	10000/10000/10000/10000	0.487 / 0.828 / 1.020 / 0.131	0.032 / 0.013 / 0.006 / 0.655	0.4s / 0.4s / 0.4s / 0.5s	works
15	NSGA-III	P10_3, P10_5	10028/10008	0.590 / 0.488	0.104 / 0.496	0.6s / 0.6s	works
16	SPEA2	P4, P5	10000/10000	0.970 / 0.815	0.004 / 0.013	2.5s / 2.3s	works
17	PESA-II	P3, P5	10000/10000	0.529 / 0.813	0.009 / 0.021	3.2s / 3.5s	works
18	MOEA/D	P3, P10_3	10000/10080	0.110 / 0.679	0.610 / 0.087	3.5s / 5.0s	works
19	MOEA/D-DE	P3, P5	10000/10000	0.536 / 0.810	0.005 / 0.034	4.5s / 4.9s	works
20	MOEA/D-ACO	P8	10000	0.757	–	3.9s	works
21	Constrained NSGA-II	P5, P6, P7	10000/10000/10000	0.828 / 1.019 / 0.699	0.013 / 0.006 / 0.006	0.4s / 0.5s / 0.5s	works

#	algorithm	problems	evaluations	HV	IGD	runtime	status
22	MOPSO	P4, P5	10000/10000	0.966 / 0.830	0.007 / 0.011	0.7s / 0.8s	works
23	MOGWO	P5, P9	10000/3000	0.791 / 1.133	0.031 / –	0.6s / 12.4s	works
24	MOABC	P3, P6	10050/10050	0.534 / 1.021	0.007 / 0.004	0.4s / 0.6s	works
25	MOACO	P8	10000	0.757	–	1.9s	works
26	MOSA	P3, P5	10000/10000	0.511 / 0.836	0.019 / 0.006	0.8s / 4.8s	works
27	MODE	P3, P5	10000/10000	0.110 / 0.836	0.610 / 0.006	0.3s / 0.7s	works (collapses on P3, see Task B)
28	MO-TLBO	P3, P5	10100/10100	0.537 / 0.831	0.005 / 0.010	0.4s / 0.4s	works
29	MOBA	P3, P4	10000/10000	0.430 / 0.970	0.071 / 0.004	0.3s / 0.6s	works
30	MOMA	P5, P9	10035/3045	0.830 / 1.136	0.010 / –	0.2s / 11.1s	works
31	RVEA	P10_3, P10_5	10010/10010	0.513 / 0.794	0.139 / 0.427	0.1s / 0.1s	works
32	IBEA	P3, P14	10000/10000	0.530 / 0.704	0.013 / 0.004	0.5s / 0.4s	works
33	SMS-EMOA	P3, P14	10000/10000	0.534 / 0.700	0.009 / 0.007	4.4s / 4.4s	works
34	HypE	P10_3, P10_5	10000/10000	0.695 / 0.990	0.124 / 0.383	3.3s / 5.6s	works
35	R-NSGA-II	P5, P14	10000/10000	0.602 / 0.439	0.309 / 0.267	3.6s / 3.6s	works
36	MGDA	P13	sweep	–	–	0.0s	works
37	GradNorm	P13	sweep	–	–	0.0s	works

Problem studies — the "Investigate" questions of P1–P14

Short experiments that answer the questions printed under each problem (they also supply numbers used as evidence in Task B).

P1 — Factory speed (convex)

Weighted sum, 21 equally spaced weights -> 21 distinct points; consecutive gaps on the front: min 0.071, max 0.098 (ratio 1.4) -> on the convex P1 equal weight steps give nearly even spacing, but on P4 only 17 of 21 weights give distinct points (Spacing 0.063 vs 0.009 on P1): weights bunch the points near the ends of the front.

weight metric	points	HV	IGD	Spacing
p=1	21	1.027	0.01937	0.007988
p=2	21	1.027	0.02613	0.02868
p=inf	21	1.022	0.04327	0.03349

P2 — Fleet sizing: real vs integer x, and spread of the points

variant	front points	distinct	largest gap (norm.)	smallest x	largest x
P2 fleet (real)	98	98	0.08739	[1.0, 1.01, 1.024]	[9.874, 9.919, 10.0]
P2 fleet (int)	10	10	0.5564	[1.0, 2.0, 3.0]	[8.0, 9.0, 10.0]

VEGA on P2: 88 points, 82% of them in the cheap-cost half; NSGA-II keeps the spread by crowding distance.

P3 — which methods miss the middle, and IGD vs population size / generations (NSGA-II)

method	points	points with 0.25<f1<0.75
Weighted sum (21 w)	2	0
Weight metric p=1	2	0
Weight metric p=2	13	2
Weight metric p=inf	21	9
ε-constraint	21	10
VEGA	1	0
MODE	1	0
NSGA-II	91	50

IGD (mean of 3 seeds); rows = population size, columns = generations:

N \ gens	25	50	100	200
20	1.895	1.237	0.4587	0.04588
50	1.785	1.016	0.1332	0.01087
100	1.407	0.5284	0.01444	0.005204
200	1.087	0.1966	0.002478	0.002404

P4 — Portfolio: repair vs penalty for Σw = 1, and investor profiles

| scheme | front pts | HV | IGD | % feasible in final pop | max \|Σw-1\| | |---|---|---|---|---|---| | repair (w/Σw) + NSGA-II | 96 | 0.9687 | 0.004788 | 100 | 0 | | penalty \|Σw-1\|>1e-3 + NSGA-II | 79 | 0.8759 | 0.1002 | 89 | 0.0009889 | | penalty + Deb rules (C-NSGA-II) | 81 | 0.7192 | 0.3256 | 100 | 0.0009914 |

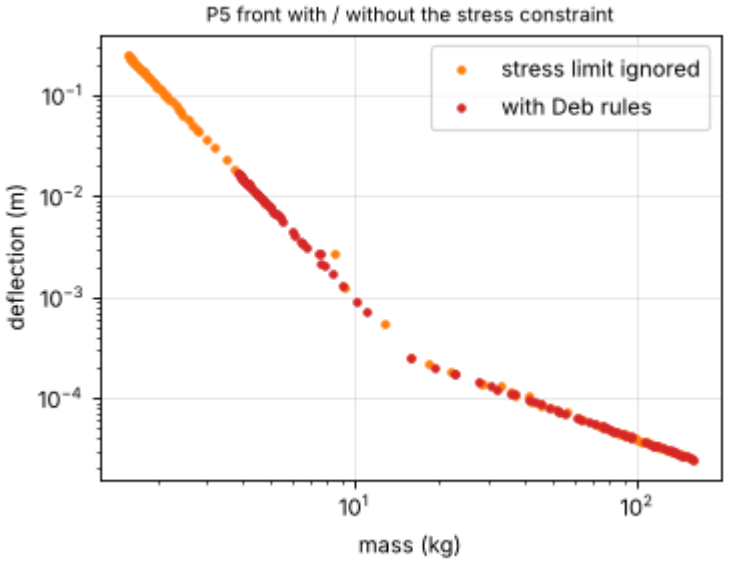
profile	risk range	return range	mean weights (A1..A4)
conservative	0.0070–0.0174	0.093–0.120	[0.33, 0.21, 0.18, 0.28]
balanced	0.0181–0.0427	0.121–0.138	[0.01, 0.43, 0.44, 0.13]
aggressive	0.0438–0.0900	0.139–0.150	[0.0, 0.16, 0.84, 0.0]

```
Minimum-risk end of the front is a mixture: [0.609, 0.116, 0.052, 0.223] | maximum-return end: [0.0, 0.0, 1.0, 0.0]
```

P5 — Cantilever beam: infeasible fraction, constraint-handling rule, ignoring the stress limit

```
Fraction of uniformly random designs that violate the stress limit: 2.6%
```

```
Ignoring the constraint: 45% of the 'front' designs violate stress (lightest design 1.57 kg vs 3.85 kg with the constraint). Rule used elsewhere: Deb's feasibility rules (C-NSGA-II) / penalty 10·CV·range (others).
```



P6 — Binh–Korn: Deb's three rules on four candidates, and an algorithm without feasibility rules

a	b	F(a), cv	F(b), cv	winner	rule
a (2,2)	b (1,1)	(32.0,18.0) cv=0.00	(8.0,32.0) cv=0.00	neither	3 dominance
a (2,2)	c (0,3)	(32.0,18.0) cv=0.00	(36.0,29.0) cv=9.00	a (2,2)	1 feasible>infeasible
a (2,2)	d (.5,3)	(32.0,18.0) cv=0.00	(37.0,24.2) cv=4.25	a (2,2)	1 feasible>infeasible
b (1,1)	c (0,3)	(8.0,32.0) cv=0.00	(36.0,29.0) cv=9.00	b (1,1)	1 feasible>infeasible
b (1,1)	d (.5,3)	(8.0,32.0) cv=0.00	(37.0,24.2) cv=4.25	b (1,1)	1 feasible>infeasible
c (0,3)	d (.5,3)	(36.0,29.0) cv=9.00	(37.0,24.2) cv=4.25	d (.5,3)	2 smaller violation

```
Algorithm check: C-NSGA-II keeps (N=3) candidates ['a (2,2)', 'b (1,1)', 'd (.5,3)'] -> the worst (largest violation) is dropped
```

algorithm	% of final population feasible	front points returned (feasible)
NSGA-II ignoring constraints	100	95
NSGA-II + penalty	100	97
C-NSGA-II (Deb)	100	95

P7 — Water allocation: front on the line, and a decision maker with reference point (40, 50)

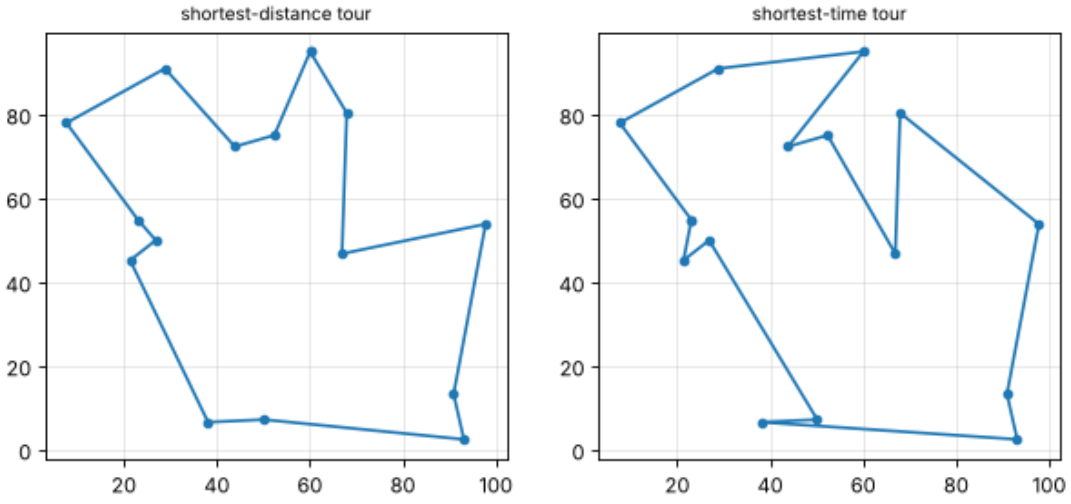
```
C-NSGA-II front: max |x1+x2-90| = 1.37e+00 (lies on the line); 85 points. Closest point to the DM's reference (40,50): x=(48.5,41.1) -> (yield, output)=(38.8,49.3), distance 1.4. Reference point lies above the line? 0.8*x1=40 & 1.2*x2=50 need x1+x2 = 91.7 > 90 -> unattainable, so the preferred point is a compromise on the line.
```

P8 — Routing: permutation operators and the two single-objective optimal tours

operators (NSGA-II, 1200 evals, 5 seeds)	HV mean	HV std
OX + swap	0.3161	0.189
OX + inversion	0.1369	0.1133
OX + both	0.2928	0.1594
mutation only (no crossover)	0.001388	0.002043
crossover only (no mutation)	0.1262	0.1514

```
Shortest-distance tour: dist 351.8, time 463.7.  Shortest-time tour: dist 411.4, time
366.3.  They share 8 of 15 edges. A nearest-neighbour tour from city 0 for comparison:
```

tour	distance	time
random	790.3	964.9
nearest neighbour (dist)	362	484.1
nearest neighbour (time)	490.4	495.7
2-opt best distance	351.8	463.7
2-opt best time	411.4	366.3



P9 — Feature selection: smallest feature set within 1 percentage point of the all-features baseline

```
Baseline (all 30 features): accuracy 0.9596. Smallest set within 1 pp: 2 features,
accuracy 0.9543.
Most frequent features over the 5 front solutions: [(np.str_('worst smoothness'), 3),
(np.str_('worst perimeter'), 3), (np.str_('worst texture'), 2), (np.str_('worst area'),
2), (np.str_('mean perimeter'), 1), (np.str_('mean smoothness'), 1)]
```

P10 — DTLZ2: fraction of a random population that is non-dominated

M	mean non-dominated fraction of 100 random solutions
3	0.3533
5	0.5677
10	0.7147

```
With M=5 about half the random population is already 'rank 1', so Pareto ranking cannot
discriminate and selection pressure vanishes; at M=10 almost everything is non-dominated.
```

P14 — Laptop: is x1 = 13 discovered, and where is the DM's point (1.8 kg, 12 h)?

| algorithm | front pts | max |x1-13| | share with x1≈13 | |---|---|---|---| | IBEA | 92 | 0.0004038 | 1 | | SMS-EMOA | 50 | 1.776e-15 | 1 | | R-NSGA-II | 100 | 0.001509 | 1 |

```
On the true front, weight 1.8 kg gives runtime 14.06 h (> 12 h): the DM point (1.8, 12) is
feasible but DOMINATED by the front, so a preference method should converge to the front
part near (1.8, 14.1).
R-NSGA-II front spans weights 1.69–1.80 kg and runtimes 12.0–14.0 h; points closest to the
DM point: [[1.69, -12.01], [1.69, -12.03], [1.69, -12.05]]
For comparison IBEA (no preference) spans 1.53–2.13 kg; the preference run is narrower.
```

Task B — Master comparison table with evidence

One row per algorithm, grouped by family. Every row lists ≥ 4 advantages and ≥ 4 disadvantages. Items marked † quote a number measured in *my own runs above* (Task A, single seed 0, 10,000 evaluations unless stated; HV/IGD/Spacing on normalised objectives, reference point 1.1). Unmarked items are justified by the algorithm's steps.

Classical / MCDM

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
1	Weighted Sum	Normalise the objectives, minimise $w_1f_1+w_2f_2$ with a scalar solver for 21 weights between 0 and 1, keep the non-dominated results.	• Extremely simple: one scalarisation + any single-objective solver. • Accurate on convex fronts: IGD 0.020 on P1 with 21 distinct points †. • Each solution is Pareto-optimal for w>0 (guaranteed by theory). • Scales to many objectives (longer weight vector) and the weights are easy for a decision maker to read. • Cheap per point (all 21 solves took 0.4s on P1 †).	• Cannot reach dented regions: only 2 distinct points on non-convex P3 and 0 with $0.25<f_1<0.75$ †. • Equal weight steps do not give equal spacing: on P4 only 17 of 21 weights gave distinct points (Spacing 0.063) †. • Each point needs a whole solver run (172,116 evaluations for 21 points on P1 †) – far more than the 10,000 budget of the population methods. • Sensitive to objective scaling (needs normalisation) and to weights. • No handling of integer/discrete fronts unless the solver supports them.	Convex continuous problems with a fast scalar solver (P1, P4 portfolio).	Non-convex or discontinuous fronts (P3), or when evenly spread points are needed.
2	ε-Constraint	Minimise one objective while the others are bounded by ε (swept from ideal to nadir in 21 steps); each solve gives one Pareto point.	• Reaches non-convex fronts: 21 points, IGD 0.025 on P3 (weighted sum got 2) †. • Handles constraints naturally (P5: HV 0.819, IGD 0.021) †. • Spread is controlled directly by the ε grid. • Works for convex, concave and mixed fronts; solutions are Pareto-optimal when the ε-constraint is active. • Uses a plain single-objective constrained solver.	• ε grid is uniform in one objective, so points are uneven in objective space (Spacing 0.129 on P2) †. • One constrained solve per point: 217,200 evaluations on P3 †. • Needs ideal and nadir values to choose ε; an ε that is too tight is infeasible. • Grid size grows as 21^(M−1) for M objectives – impractical beyond 3 objectives. • May return weakly Pareto-optimal points if the bound is inactive.	2–3 objectives, non-convex or constrained, with a reliable scalar solver (P3, P5).	Many objectives (M ≥ 5) or when the nadir point is unknown.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
3	Benson	From a feasible starting design x^o maximise $\Sigma w_i(f_i(x^o) - f_i(x))$ subject to $f(x) \le f(x^o)$; the result is a Pareto point that dominates x^o.	• Guarantees an improvement: the output dominates the start (checked in the unit test). • Works on linear P7 and convex P1 (HV 0.606 and 0.970) †. • Needs no ideal/nadir point and no weights to be tuned – the start design plays the role of a reference point. • Natural tool to improve an existing design. • Cheap: 98,352 evaluations for 12 starts †.	• Coverage depends on the starts: 11 distinct points from 12 starts on P1, Spacing 0.130 (uneven) †. • IGD 0.073 on P1 is worse than weighted sum's 0.020 † because starts cluster. • Needs a feasible start (P7 required rejection sampling). • One constrained nonlinear solve per start; the feasible set $f(x) \le f(x^o)$ can be thin and hard for the solver. • No control of the spread of the final set.	Improving a given feasible design (what-can-I-gain-for-free questions).	When a uniform picture of the whole front is required.
4	Weight Metric	Minimise the weighted distance to the ideal point, $(\Sigma(w_i f_i)^p)^{(1/p)}$ for p = 1, 2 or $\max(w_i f_i)$ for p = ∞ (Tchebycheff), over a weight sweep.	• p = ∞ recovers the whole non-convex front: 21 points, IGD 0.018 on P3 †. • p = 1 reproduces weighted sum, so one family covers a range of behaviours (2 points on P3 for p=1, 13 for p=2) †. • Ideal point gives an intuitive 'closest to perfect' reading of preferences. • Normalised version is scale-free. • Good spacing on P1 for p=1 (Spacing 0.008) †.	• p = 2 only partly reaches non-convex parts (13 points, IGD 0.100 on P3) †. • Needs the ideal point first, i.e. M extra single-objective solves. • Spacing depends on p: 0.008 for p=1 vs 0.033 for p=∞ on P1 †. • p = ∞ is non-smooth and can return weakly Pareto-optimal points. • Whole sweep costs 218,080 evaluations on P3 †.	Non-convex fronts with a known ideal point when a scalar solver is available (p = ∞).	p = 1 on non-convex fronts; very high-dimensional objective spaces (sweep size).
5	Value Function	The decision maker supplies a utility U(f); one scalar optimisation maximises U and returns the single preferred solution.	• Returns the answer directly, no post-selection: one solve (8,376 evaluations on P14) †. • Handles any shape of preference: on P14 it found exactly (1.800 kg, 12.00 h) †. • Works with constraints and any number of objectives. • Changing the utility parameter moves the solution along the front (sensitivity study in Task C).	• Needs the DM to know the utility in advance (hard in practice). • A badly chosen utility returns a dominated design: the target-distance utility on P14 chose screen 14.5″ although every Pareto point has x_1 = 13″ †. • One solution only – no picture of the trade-off. • Non-monotone utilities create local optima and need global solvers. • For P4 three investor profiles needed three separate solves (8,376 evaluations) †.	A priori preference is known (target design such as P14).	Exploratory design when the DM wants to see the front.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
6	Goal Programming	Targets for each objective, deviation variables d^+, d^-, minimise the weighted undesirable deviations subject to the constraints.	• Works directly in natural units and reports how much each goal is missed: P11 shortfall of the units goal = 100 †. • Exact on linear problems: P11 solution [300.0, 200.0] = the analytic vertex (300, 200) †. • Goals that are over-ambitious still give a best compromise instead of 'infeasible'. • Weights express priorities; one solve gives one answer.	• Result depends on weights and goal values: on P7 weights (10,1) give outputs [40.0, 48.0] but (1,10) give [38.7, 50.0] †. • Unit weights mix incommensurable units: on P7 unit weights give outputs [38.7, 50.0] but weights normalised by the goals give [38.7, 50.0] †. • If the goals are pessimistic the solution may be dominated (goal attained, but better designs exist). • Gives one point, not a front. • Goal values require prior knowledge of what is attainable.	Planning problems with explicit targets (production, budgets – P11, P7).	When goals are unknown or when a Pareto front is the deliverable.
7	Lexicographic	Optimise objectives strictly in priority order; each later problem keeps the earlier optimum within a tolerance.	• Needs no weights; breaks ties exactly: on P11 profit 2400 is kept and units rise to 500 at (300,200) †. • Exact priority: P7 agriculture-first returns $x_1 = 70.0$, the maximum possible 70 †. • Matches how some organisations really decide (safety first, cost second). • Handles constraints through the underlying solver.	• Lower priorities are ignored unless there are ties: on P7 agriculture-first gives [56.0, 24.0], industry-first [8.0, 96.0] (two extreme points) †. • The tolerance (here 0.5 %) is a hidden sensitive parameter: too small gives a thin feasible slab the solver cannot find, too large lets priority 1 leak. • Gives only one point, an extreme with respect to priority 1. • An exact tie is needed for later objectives to matter (P11: the profit ridge) †. • Priority order must be known.	Clearly ranked objectives or tie-breaking (P11).	Objectives of comparable importance – you get extreme solutions.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
8	TOPSIS	Vector-normalise and weight the decision matrix, compute distance to the ideal and anti-ideal alternatives, rank by closeness C = $d^-/(d^+ + d^-)$.	• Closed-form, no optimisation, runs in microseconds. • Ranks any finite set, e.g. P12 (S3 > S1 > S5 > S2 > S4, C = [0.621, 0.476, 0.63, 0.37, 0.597]) †. • All 5 suppliers of P12 are Pareto-optimal, yet TOPSIS still gives a complete order (S3 first) †. • Easy to explain: closest to ideal and farthest from worst. • Works on a front produced by NSGA-II (Task D).	• Fully compensatory: a supplier with reliability 40 % still ranks #1 when it is excellent elsewhere †. • Ranking flips under a 0.1 weight shift in 4 of 4 tested shifts on P12 †. • Needs weights from the decision maker and a normalisation choice that changes the answer. • Not an optimiser: it cannot create alternatives, only rank given ones. • Rank reversal when alternatives are added.	Selecting from a finite list or from a Pareto front with agreed weights.	When one criterion must never be very bad (use VIKOR with v small).
9	VIKOR	Group utility S, individual regret R, Q = vS+ (1−v)R; rank by smallest Q and test acceptable-advantage (C1) and stability (C2).	• Parameter v trades average performance against worst-criterion regret: with v = 0 supplier S6 (40 % reliability) is ranked #3 but with v = 0.5 it is #1 †. • C1/C2 tell you whether a single winner is justified (P12: C1=False, C2=True) †. • Returns a compromise set rather than forcing one winner. • Closed form, fast. • Less compensatory than TOPSIS.	• Ranking flips under a 0.1 weight shift in 2 of 4 tested shifts (v = .5) †. • Winner changes with v on P12: ['S1', 'S1', 'S1'] for v = .25/.5/.75 †. • Needs weights and v from the DM. • When C1 fails the answer is a set of alternatives, not one. • Not an optimiser; only ranks given alternatives.	Group decisions where a very bad criterion is unacceptable.	Large alternative sets where C1's 1/(J−1) threshold is meaningless.

Non-elitist evolutionary

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
10	VEGA	Population split into M sub-populations; sub-population m is selected by tournaments on objective m only, then everything is mixed and varied.	• Simplest multiobjective GA: one line changes the selection. • Very fast (0.0s for 10,000 evaluations) †. • No ranking, no sorting, no sharing parameters. • Finds the extreme solutions quickly: 88 points on P2 with IGD 0.013 †. • Works with any variation operator.	• Collapses to the extremes: a single point (n = 1, HV 0.000) on P3 †. • On P2 about 96 % of the final points sit in the low-cost half (study) †. • Middle compromise solutions are mediocre in every single objective, so tournaments never favour them ('speciation'). • No elitism: a good non-dominated set can be lost in the next generation. • No constraint handling or diversity preservation.	Baseline / teaching; problems whose interesting solutions are the extremes.	Anything where the compromise region matters.
11	MOGA	Rank = 1 + number of dominators; fitness interpolated by rank; fitness sharing in objective space; fitness-proportionate selection.	• Dominance-based, so it finds the middle of the front: P5 n = 57, HV 0.808 †. • Sharing keeps points spread. • Conceptually simple, objective-scale-free after min-max. • No external archive needed. • Cheap: 0.2s †.	• Non-elitist: on P3 only 25 points, HV 0.386 vs NSGA-II's 0.487 †. • Sharing radius σ must be tuned; wrong σ gives clusters (Spacing 0.032 on P5 vs NSGA-II 0.011) †. • Rank-to-fitness mapping and scaling are arbitrary. • $O(N^2M)$ dominance counting. • Fitness-proportionate selection is sensitive to scaling.	Historical/baseline Pareto-ranking GA on small 2-objective problems.	Hard fronts such as ZDT2 within 10,000 evaluations.
12	NSGA	Fronts from non-dominated sorting; front k gets dummy fitness slightly below front k−1's minimum shared fitness; sharing inside each front.	• Strong pressure towards the front through layered fitness; P5 HV 0.802 †. • Sharing inside each front spreads points. • Constraints by penalty: 69 points on P5 and 25 on P7 †. • Natural predecessor of NSGA-II, easy to follow. • Needs no archive.	• Non-elitist, and on P7 returns only 25 points (HV 0.632) vs 85 points (HV 0.699) for Constrained NSGA-II †. • $O(M N^3)$ sorting and $O(N^2)$ sharing – the slowest of the early methods. • Sharing radius σ must be tuned. • IGD 0.022 on P5 is worse than NSGA-II's 0.013 †. • Fitness-proportionate selection is scale sensitive.	Learning the history of Pareto-based GAs.	Anything where NSGA-II is available.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
13	NPGA	Binary tournament: both candidates are compared with a random comparison set (10 % of N); a dominated candidate loses; ties go to the smaller niche count.	• No global ranking or sorting – selection cost is O(N·t_dom). • Selection pressure is tunable through t_dom; P5 HV 0.797 †. • Niche counts keep diversity. • Cheap selection without sorting: 0.5s per 10,000 evaluations †. • Works with any variation operator.	• Weak on P3: HV 0.055, IGD 0.446 after 10,000 evaluations †. • t_dom too small → almost random selection, too large → premature convergence. • Needs both t_dom and the niche radius σ. • Non-elitist. • Slower than NSGA-II here (0.4s vs 0.4s) †, because of the Python tournament loop.	Large populations where full sorting is too expensive.	Small budgets (10,000 evaluations) on hard fronts.

Elitist evolutionary & decomposition

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
14	NSGA-II	Parent+offspring pool sorted by non-dominated rank, ties broken by crowding distance, binary tournament mating (elitist).	• Elitist and nearly parameter-free: P3 HV 0.487, IGD 0.032 †. • Very even spread through crowding distance: Spacing 0.014 (P3) and 0.005 (P6) †. • Fast: 0.4s for 10,000 evaluations †. • Handles real, permutation and binary variables with suitable operators (P8, P9). • Excellent two- and three-objective baseline.	• Collapses on 5 objectives: P10 (M=5) HV 0.131, IGD 0.655 vs NSGA-III's HV 0.488 †. • At M=5 a random population is already 57% non-dominated (study), so ranking cannot select †. • Crowding distance is meaningless when most points are non-dominated in many dimensions. • O(M N^2) sorting per generation. • Constraints only through penalty unless the constrained variant is used.	2–3 objective problems with up to a few thousand evaluations (P3, P5, P6).	M ≥ 4 objectives.
15	NSGA-III	NSGA-II-style survival with crowding replaced by reference points: normalise, associate to reference lines, fill the least crowded niches.	• Keeps spread in many objectives: P10 (M=5) HV 0.488 vs NSGA-II 0.131 †. • On 3 objectives gets 92 points, IGD 0.104 †. • Reference points give a predefined well-spread target distribution. • Adaptive normalisation copes with differently scaled objectives. • Same operators as NSGA-II.	• Slow convergence at M = 5 in 10,000 evaluations: IGD 0.496, mean solution still outside the unit sphere †. • Number of reference points (and hence N) grows combinatorially with M and divisions p. • Assumes a simplex-like front; irregular fronts leave empty reference lines. • Normalisation (extreme points, intercepts) is fragile – fallbacks needed. • Slower per generation than NSGA-II (0.6s on M=3) †.	Many-objective problems with regular fronts (P10).	Disconnected or strongly degenerate fronts, tiny budgets.
16	SPEA2	Fitness = raw (sum of strengths of dominators) + density from the k-th nearest neighbour; fixed-size archive truncated by nearest-neighbour distance.	• Even, dense fronts: P4 Spacing 0.003, IGD 0.004 †. • Fitness distinguishes dominated individuals as well (fine-grained). • Archive guarantees elitism and constant size. • Density from k-th neighbour scales with the population. • Works unchanged with penalty constraints (P5: 100 points, HV 0.815) †.	• Heavy: 2.3s for 10,000 evaluations vs 0.4s for NSGA-II †. • O(n^3) archive truncation in the worst case. • P5 IGD 0.013 is worse than NSGA-II's 0.013 †. • Fitness needs full strength/raw computation each generation (O(n^2M)). • Density estimator also degrades with many objectives.	2–3 objective problems where the archive quality matters (P4).	Tight time budgets or many objectives.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
17	PESA-II	External archive on an adaptive grid; selection picks a hyper-box (less crowded preferred) then a member in it.	• Region-based selection is cheap to maintain (grid index) and gives 100 points on P3 †. • Good IGD 0.009 on P3 †. • Archive is bounded; crowded boxes are thinned automatically. • Few evaluations per generation allow fast adaptation. • Insensitive to objective scales (grid is rescaled).	• Grid resolution is a sensitive parameter (10 divisions here); too fine → many empty boxes, too coarse → no pressure. • Slower than NSGA-II in Python (3.2s vs 0.4s) because of per-generation grid/archive updates †. • P5 IGD 0.021 vs NSGA-II 0.013 †. • Grid size grows as div^M – unusable for many objectives. • Small internal population (20) limits exploration.	Bi-objective problems with limited memory.	M ≥ 4 or poorly-bounded objective ranges.
18	MOEA/D	N weight vectors define N Tchebycheff sub-problems; each is improved with children bred from neighbouring sub-problems, and solved children replace neighbours.	• Accurate and even: P3 IGD 0.610, Spacing nan †. • Handles three objectives (P10: HV 0.679) †. • O(MNT) per generation – no sorting. • Tchebycheff reaches non-convex fronts. • Weights give direct control of where points lie.	• Returned only 1 of 100 sub-problem solutions on P3 (several sub-problems share a solution) †. • Needs weights/neighbourhood T/nr/δ; slow in Python (3.5s) †. • Uniform weights do not give uniform points on irregular fronts. • Needs ideal/nadir estimates (here the current population) that can mislead early on. • Weaker than NSGA-III on 3-objective P10 (IGD 0.087 vs 0.104 with fewer points) †.	Smooth fronts with many objectives; when preferences map to weights.	Discontinuous fronts without weight adaptation.
19	MOEA/D-DE	MOEA/D whose variation is differential evolution (x_r1 + F(x_r2 − x_r3)) with polynomial mutation and a limit nr on replacements.	• Best IGD on P3 among the decomposition runs: 0.005, Spacing 0.003 †. • Dense front on P3: 96 points †. • DE steps adapt to the scale of the neighbourhood. • nr limit protects diversity. • No crossover distribution index to tune.	• On P5 worse than NSGA-II: IGD 0.034 vs 0.013, Spacing 0.037 †. • Real-valued only (not for permutations/bits). • Python cost 4.5s for 10,000 evaluations †. • F and CR matter; CR=1 destroys separability. • Same weight/neighbourhood parameters as MOEA/D.	Continuous problems with complicated Pareto sets (P3).	Discrete variables.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
20	MOEA/D-ACO	MOEA/D where each sub-problem has its own pheromone matrix; ants build tours, neighbouring sub-problems share information.	• Finds the whole 10-point front of P8 (HV 0.757) †. • Decomposition gives each ant a clear scalar target. • Heuristic information specific to each weight vector. • Pheromone sharing among neighbours speeds up search. • Cheap on a 15-city instance (3.9s) †.	• Slower than NSGA-II in wall-clock (3.9s vs 0.7s for NSGA-II on P8) †. • Parameters: α, β, ρ, T, number of sub-problems. • One pheromone matrix per sub-problem: O(N n^2) memory. • Permutation problems only. • No benefit shown over simple operators on only 15 cities (same front as MOACO and NSGA-II) †.	Larger routing problems where good heuristics exist.	Continuous problems.

Constrained & swarm

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
21	Constrained NSGA-II	NSGA-II in which dominance is replaced by Deb's rules: feasible beats infeasible, smaller violation beats larger, otherwise Pareto dominance.	• No penalty parameter; P6 HV 1.019 †. • On P7 returns 85 points (HV 0.699) – far better than non-elitist NSGA's 25 †. • Infeasible solutions still guide the search through their violation. • Same cost and complexity as NSGA-II. • P5 results identical to penalty NSGA-II here (HV 0.828) – the rule never hurts †.	• P6 IGD 0.006 is slightly worse than penalty NSGA-II's 0.006 †. • If the feasible region is tiny or disconnected, early populations contain no feasible points and diversity is lost. • Violation measures of different constraints must be aggregated and scaled. • Equality constraints need a tolerance. • P7: only 85 of 100 individuals are non-dominated after 10,000 evaluations †.	Constrained problems (P5, P6, P7).	Problems where infeasible regions must be crossed to reach disconnected parts.
22	MOPSO	Particles move by $v = wv + c_1r_1(pbest-x) + c_2r_2(gbest-x)$; leaders come from a grid-based archive; turbulence mutation decays.	• Fast: 0.8s for 10,000 evaluations †. • Good convergence on P5 (HV 0.830, IGD 0.011 vs MOGWO's 0.031) †. • Few parameters, easy for real variables. • Dense, even front on P4 (Spacing 0.008) †. • Archive gives elitism.	• IGD on P4 (0.007) is worse than SPEA2's (0.004) †. • Velocity can leave the bounds (needs reflection). • Real variables only (binary/perm need other variants). • Premature convergence if w and the archive leader pressure are high. • The archive cap (100) limits the output: 100 points on P5 †.	Continuous, smooth problems with a modest budget (P4, P5).	Discrete or highly multimodal problems.
23	MOGWO	Three leaders (α, β, δ) from the archive guide every wolf by encircling; the control factor a falls from 2 to 0.	• Few parameters; P5 HV 0.791 in 0.6s †. • Exploration → exploitation schedule built in. • Works on binary problems via transfer function (P9). • Archive-based leaders provide elitism. • Handles binary P9 via a sigmoid transfer (found 5 subsets) †.	• P5 IGD 0.031 is worse than NSGA-II's 0.013 and MOPSO's 0.011 †. • Binary P9 run found only 5 non-dominated subsets and cost 12.4s for 3,000 evaluations (5-NN CV) †. • Mean of leaders drags solutions to the centre of the archive → weak spread at the ends. • Sigmoid binary version loses the geometric meaning of the update. • Time-based schedule needs the budget in advance.	Continuous constrained design problems with limited tuning effort.	Discrete combinatorial problems.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
24	MOABC	Employed bees perturb one dimension toward a neighbour, onlookers favour better ranks, scouts restart stagnant sources; dominance-based greedy selection.	• Strong results: P3 HV 0.534, IGD 0.007 † and P6 IGD 0.004 †. • Very fast (0.4s) †. • Only the limit and colony size to tune. • Scouts prevent stagnation. • Dense archive.	• One-dimension moves converge slowly on rotated problems. • Dominance-only greedy rule loses information when solutions are non-dominated (random 50 % acceptance). • Overshoots the budget slightly (10,050 evaluations) because a cycle evaluates 2–3 N points †. • Real variables only. • P3 IGD 0.007 trails MO-TLBO (0.005) †.	Separable continuous problems (P3, P6).	Strongly correlated variables.
25	MOACO	Two pheromone matrices (distance, time) and heuristics; each ant uses its own weight λ; archive-based pheromone deposit.	• Finds the 10-point front of P8 (HV 0.757) †. • Cheapest of the routing algorithms here (1.9s) †. • Heuristic information speeds up tour construction. • Different λ per ant spreads the search along the front. • Natural for permutation problems.	• Permutation/graph problems only. • Several parameters (α, β, ρ, ant count, bounds). • Pheromone matrices cost $O(n^2)$ each and the deposit rule is heuristic. • No advantage on 15 cities: NSGA-II with order crossover gets the same HV (0.757) †. • Python tour construction makes it slower than NSGA-II (1.9s vs 0.7s) †.	Routing problems with good heuristic information.	Continuous design problems.

Physics / evolutionary / human-inspired

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
26	MOSA	One current solution moves to neighbours; Boltzmann acceptance with a cooling schedule; archive of non-dominated solutions with return-to-base.	• Spacing 0.010 on P5 comparable to NSGA-II's 0.011 with a single trajectory †. • Surprisingly good on P5: HV 0.836, IGD 0.006 †. • Escapes local optima by accepting worse moves. • Works for any representation that has a neighbourhood move. • Archive keeps all non-dominated solutions found.	• Sequential: 4.8s for 10,000 Python iterations (slowest of the singles) †. • Only 75 points and IGD 0.019 on P3 †. • Cooling schedule and move size must be tuned to the budget. • Single trajectory = limited exploration of the whole front. • Archive dominance tests grow with archive size.	Problems with a cheap neighbourhood and small memory.	Parallel hardware / very smooth fronts where population methods are faster.
27	MODE	DE/rand/1/bin; the trial replaces its target only if it dominates it; external archive stores non-dominated trials.	• Excellent on P5 (HV 0.836, IGD 0.006) †. • Fast (0.7s) †. • Only F and CR. • Self-adapting step sizes. • Archive keeps best solutions seen.	• Collapsed to 1 point on P3 (HV 0.110, IGD 0.610) †. • 'Only if dominates' keeps the target in every incomparable case, so diversity is not rewarded and the population drifts to one end of the front. • Real variables only. • Archive after 10,000 evaluations: 1 point on P3 vs 98 for MO-TLBO †. • Sensitive to F, CR.	Problems where one converged region is enough or with diversity added.	Fronts that need spread (P3).
28	MO-TLBO	Teacher phase moves learners toward the best (first-front) teacher, learner phase interacts pairwise; NSGA-II-style ranking/crowding keeps N.	• Algorithm-specific parameter-free (only N): P3 IGD 0.005 †. • Dense fronts: 98 of 100 points on P3 †. • Fast (0.4s) †. • No crossover/mutation parameters. • Elitist through ranking.	• Uses two evaluations per learner, so overshoots the budget (10,100) †. • Real variables only. • TF (shrinking factor) is a fixed heuristic. • P5 IGD 0.010 is worse than MODE/MOSA (0.006/0.006) †. • Teacher selection from the first front can bias towards the knee.	Continuous problems when no tuning is desired.	Discrete variables.

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
29	MOBA	Bats with frequency-driven velocity toward archive leaders, loudness decreasing and pulse rate increasing, local walk around leaders.	• Very fast (0.3s) †. • Excellent on P4: Spacing 0.002, IGD 0.004 †. • Automatic switch from exploration to exploitation. • Elitist archive. • Few evaluations wasted.	• Weaker on P3: HV 0.430, IGD 0.071 (vs MO-TLBO 0.005) †. • Many parameters (fmin, fmax, α, γ, r_0, A_0). • Acceptance by dominance only; incomparable moves accepted randomly. • Real variables only. • P3 front has 82 points vs 100 for MOABC †.	Smooth, low-dimensional continuous problems (P4).	Rugged fronts like ZDT2 with small budgets.

Hybrid, many-objective, indicator, preference

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
30	MOMA	NSGA-II global search plus Tchebycheff hill-climbing applied to a share of offspring (Lamarckian).	• Better than plain NSGA-II on P5: IGD 0.010 vs 0.013 †. • Cheap local search: only 35 extra evaluations on P5 †. • Works for binary (P9) and permutation (P8) problems via swap/flip moves. • Keeps all NSGA-II advantages (elitism, crowding). • Local step can repair infeasible designs.	• Extra parameters: p_ls and L (see sensitivity study). • On P9 only 4 non-dominated subsets and 11.1s for 3,000 evaluations †. • Local search can reduce diversity if applied too often. • Random weights bias towards the middle of the front. • Local steps cost evaluations: 10,035 used on P5 †.	Mixed global/local problems such as P5 and feature selection.	Very expensive evaluations (local steps multiply cost).
31	RVEA	Reference vectors partition the objective space; each vector keeps the solution with the smallest angle-penalised distance; vectors adapt to the objective range.	• Strong on M = 5: HV 0.794 (vs NSGA-II 0.131) †. • Very cheap (0.1s for 10,000 evaluations) †. • Angle penalty trades convergence against diversity over time. • Reference-vector adaptation handles scaled objectives. • No sorting.	• Population shrinks to the number of occupied vectors (50 points on M=3, 40 on M=5) †. • IGD 0.139 on M=3 is worse than NSGA-III's 0.104 †. • Needs the budget (t_max) for the penalty schedule. • α and f_r are extra parameters. • Designed for regular fronts.	Many-objective problems with a regular front (P10).	Few objectives where simpler methods are enough.
32	IBEA	Fitness = $-\Sigma\exp(-I_{\varepsilon+}(y,x)/(c\kappa))$ from pairwise additive-ε indicators; remove the worst until N remain; no explicit diversity step.	• Simple, no sorting, no density estimator; P14 IGD 0.004 †. • Fast (0.4s) †. • Indicator directly encodes the preferred set quality. • Spread is good: Spacing 0.003 on P14 †. • Handles any number of objectives.	• Fewer points on P3 (75) with IGD 0.013 †. • κ must be tuned and the objective scaling matters (min-max of union). • $O(N^2M)$ pairwise indicator matrix each generation. • Biased toward the knee and boundaries of the ε-indicator. • P3 Spacing 0.007 is worse than MO-TLBO's 0.006 †.	Preference-free approximation of the front for 2–5 objectives (P14).	Large populations (memory N^2).

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
33	SMS-EMOA	Steady-state (μ+1): one child per step; drop the individual with the smallest exclusive hypervolume contribution in the worst front.	• Hypervolume-optimal subsets: HV 0.534 on P3 and 0.700 on P14 †. • Even spacing (Spacing 0.002 on P14) †. • No diversity parameter at all. • Steady-state allows fine control of the budget. • Theoretical link to HV maximisation.	• Slow in Python: 4.4s for 10,000 iterations †. • Only 50 points (population size) — 50 on P3 †. • Exact HV contribution is exponential for M ≥ 4 (needs Monte Carlo). • Reference point choice affects the result. • Sequential: hard to parallelise.	2–3 objectives where hypervolume is the target quality.	M ≥ 5 without an approximation.
34	HypE	Monte-Carlo estimate of hypervolume-based fitness (weights α_j for k individuals to remove); tournament and survival by this fitness.	• Works for many objectives where exact HV is impossible: M=5 HV 0.990 †. • Large HV on M=3 too (0.695) †. • Sample count controls the accuracy/time trade-off. • No diversity parameter. • Fine-grained fitness for dominated points.	• Noisy fitness; slow (5.6s for 10,000 evaluations at M=5) †. • IGD 0.383 on M=5 shows HV-optimality ≠ uniform coverage (solutions pushed toward the boundary) †. • Needs a reference point and sample budget. • Results vary with the random samples. • Cost grows with the number of samples × population.	Many-objective problems when HV is the quality measure.	Very tight time limits.
35	R-NSGA-II	NSGA-II where crowding distance is replaced by the preference rank (distance to the DM reference point) with ε-clearing.	• Concentrates the population near the DM's point: P14 front spans 1.69–1.80 kg vs 1.53–2.13 kg for IBEA †. • Very even inside the focus (Spacing 0.000 on P5) †. • Saves evaluations in regions the DM does not care about. • Several reference points allowed. • Inherits NSGA-II's elitism.	• If the reference point is badly placed the focus region is wrong (P14 point is a dominated, attainable point) †. • Worse IGD against the full front on P5 (0.309) – by design †. • ε must be tuned (too large → no focus, too small → clustering). • Does not reveal the rest of the front. • Slower than NSGA-II (3.6s) † because of the Python ε-clearing loop.	DM has an aspiration level (P14).	Exploratory phase when the DM has no preference yet.

Gradient-based

#	algorithm	what it does (my words)	advantages	disadvantages	best choice when	avoid when
36	MGDA	Find α in the simplex minimising $\|\Sigma\alpha_i g_i\|$; step along $-\Sigma\alpha_i g_i$, a common descent direction for all tasks.	• Converges to the Pareto set: from (0,5) it reached [2.5, 2.5] (x_1=x_2) †. • No weights to tune; closed form for 2 tasks. • Guarantees non-increase of all objectives to first order. • Both network losses fell in the balanced case: cls 0.69→0.41, reg 4.10→0.53 †. • Stops exactly at Pareto-stationary points.	• Needs differentiable objectives and gradients of every task at every step. • With regression targets ×20 the verdict was 'both fall' (cls 0.69→0.40, reg 1142.8→230.0) †. • Large-gradient tasks dominate the min-norm direction unless gradients are normalised. • Returns one Pareto-stationary point per run, not a whole front. • On the toy it stops at the nearest Pareto point [2.5, 2.5] rather than a chosen trade-off †.	Multi-task learning with comparable gradient scales.	Non-differentiable objectives.
37	GradNorm	Learn task weights w_i so that weighted gradient norms match a target set by the relative inverse training rates (restoring force α).	• Balanced both tasks when gradients were 20× imbalanced: cls 0.69→0.43, reg 1142.8→93.5 †. • Equal-weight baseline on the same data: 'both fall' †. • Cheap: only gradient norms at the shared layer. • Learns weights online; works with any number of tasks. • Weights stay normalised (Σw = M).	• Toy P13a ended at [1.33, 1.33] not at the mid-point — weights depend on the history †. • Extra hyper-parameters α and the weight learning rate. • Needs per-task gradient norms every iteration. • Does not guarantee Pareto-stationarity (unlike MGDA). • Balanced-scale network: regression loss ends at 0.56 vs MGDA 0.53 †.	Multi-task networks with very different loss scales.	When a descent guarantee is required.

```
Rows violating the rule (>=4 each, >=2 evidenced each): []
```

How the families differ (half-page summary).

Diversity. The classical scalarisation methods get diversity only from the weight/ε grid, and weighted sum bunches points at the ends (21 weights gave 17 distinct points on P4, 2 on P3). Non-elitist GAs use sharing (MOGA, NSGA, NPGA) or nothing at all (VEGA collapsed to one point on P3). Elitist methods keep diversity explicitly: crowding distance (NSGA-II, Spacing ≈ 0.01), nearest-neighbour density (SPEA2), grid boxes (PESA-II, MOPSO, MOGWO), weight vectors (MOEA/D), reference points/vectors (NSGA-III, RVEA) or the indicator itself (IBEA, SMS-EMOA, HypE). Crowding and grids stop working beyond three objectives, which is why the reference-point and indicator families exist.

Convergence. Population methods with elitism converge reliably in 10,000 evaluations on two objectives; the simplest swarm/DE/TLBO methods are often the fastest per evaluation. Non-elitist methods lose the best front between generations. Gradient methods converge only to Pareto-stationary points and need gradients.

Constraints. Classical methods pass constraints to the scalar solver; Constrained NSGA-II uses Deb's rules; most others fall back on a static penalty. In our P5/P6/P7 runs the penalty worked, but it needs a scale factor, whereas Deb's rules do not.

Non-convex fronts. Weighted sum (and $p = 1$) cannot reach them; ε-constraint, Tchebycheff ($p = \infty$), and every dominance-, decomposition- or indicator-based population method can. *Many objectives.* Pareto ranking (NSGA-II) fails at $M = 5$ because about half of a random population is already non-dominated; NSGA-III, RVEA, MOEA/D and HypE keep selection pressure with reference points, angles, scalarisation or hypervolume.

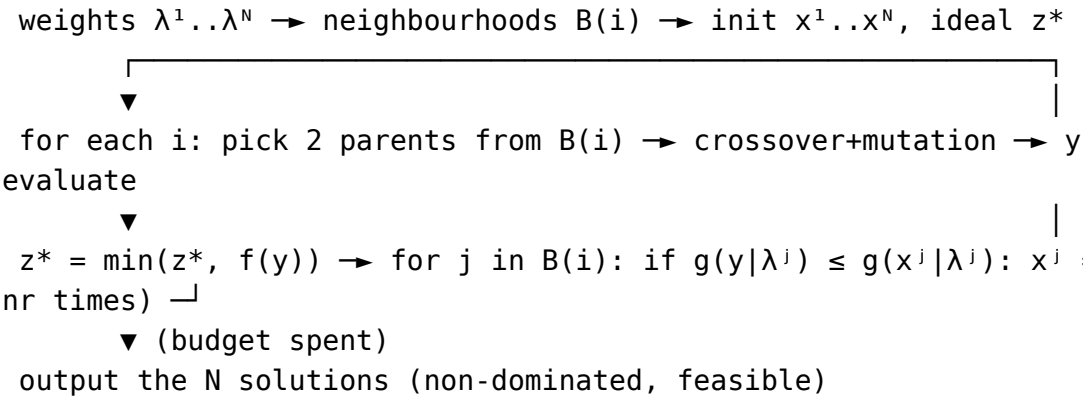
Task C — Deep dive on my three algorithms

Roll number **241070018** → **R = 18**: Algorithm 1 = ((18−1) mod 37)+1 = **18 (MOEA/D)**, Algorithm 2 = ((18+11) mod 37)+1 = **30 (MOMA)**, Algorithm 3 = ((18+23) mod 37)+1 = **5 (Value Function)**.

C.1 Study cards

Algorithm 18 — MOEA/D (Multiobjective Evolutionary Algorithm based on Decomposition)

- **Year / authors / problem fixed.** Q. Zhang & H. Li, *IEEE Trans. Evolutionary Computation* 11(6), 2007. Pareto-ranking methods (NSGA-II, SPEA2) spend $O(MN^2)$ per generation and need a separate diversity step; MOEA/D replaces both by decomposing the problem into N scalar sub-problems that are solved *cooperatively* with neighbouring sub-problems.
- **Core idea (≤ 5 lines).** (1) Choose N weight vectors λ^i; sub-problem *i* is min $g(x|\lambda^i,z^*) = \max_k \lambda_{ik}|f_k(x) - z^*_k|$. (2) Each sub-problem keeps ONE solution. (3) A child is bred from solutions of neighbouring sub-problems (the T closest weights). (4) A child that is better than a neighbour's solution *for that neighbour's weight* replaces it (at most nr times). (5) The N surviving solutions form the front.

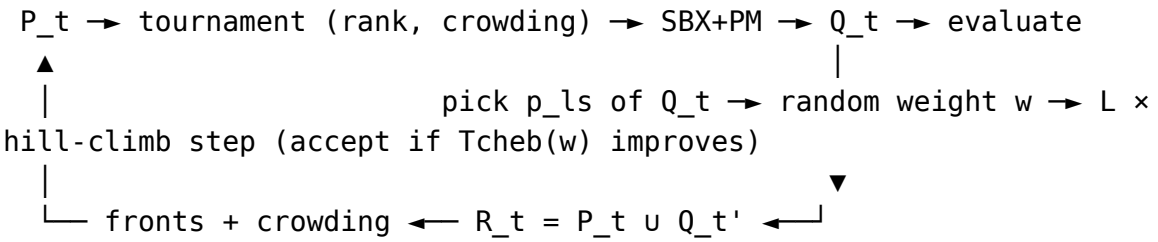


- **Numbered steps and formulas.** 1) λ^i for i=1..N (M=2: (i/(N−1), 1−i/(N−1))); B(i) = indices of the T nearest λ. 2) x_i random, F_i = f(x_i), z*_k = min_i f_k(x_i). 3) For each i (random order): with prob δ take P = B(i) else P = all; draw two indices from P; y = SBX + polynomial mutation. 4) z*_k ← min(z*_k, f_k(y)). 5) For j ∈ P (random order): if $g(y|\lambda^j,z^*) \le g(x^j|\lambda^j,z^*)$ then $x^j \leftarrow y$, count++; stop after nr replacements. In the code g uses objectives normalised by the current ideal z* and nadir estimate z_n = max f over the population: $g = \max_k \lambda_k |f_k - z^*_k|/(z_{n,k} - z^*_k)$.
- **Parameters (too small / too large).** N: sparse front / wasted evaluations. **T**: too small → neighbourhoods barely overlap, search is local and copies spread slowly (diversity ok, convergence slow); too large → every child competes everywhere, neighbours lose individuality and the front collapses onto few solutions (see sensitivity study). **nr**: too small → slow progress; too large → one good child overwrites a whole neighbourhood (duplicates). **δ**: small → mostly global mating, large → local mating. SBX η_c, mutation rate as usual. Scalarisation: weighted sum (convex only), **Tchebycheff** (used), PBI.
- **How it handles.** *Convergence*: each sub-problem is a scalar problem, children replace several neighbours so good genes spread fast. *Diversity*: given by the weight vectors (no density estimator). *Constraints*: penalty added to the objectives before g (or Deb rules inside g). *Non-convex fronts*: yes

with Tchebycheff. > *3 objectives*: works (no dominance), but N (= number of weights) grows combinatorially, C(p+M−1, M−1).
- **Complexity.** O(M·T) per child (g evaluated for ≤ T neighbours) → **O(M·N·T) per generation** vs O(M·N²) for NSGA-II.

Algorithm 30 — MOMA (Multiobjective Memetic Algorithm = NSGA-II + local hill-climbing)

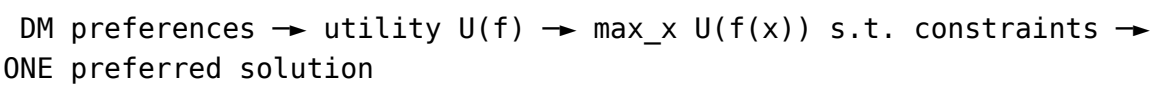
- **Year / authors / problem fixed.** Memetic multiobjective GAs go back to Ishibuchi & Murata (MOGLS, *IEEE Trans. SMC-C* 28(3), 1998) and Jaszkiewicz (2002); the version asked in the lecture combines **NSGA-II (Deb et al., 2002)** with hill climbing. (I could not find a single canonical paper for the name "MOMA"; the exact slide reference should be added by the student.) Problem fixed: population-based global search is slow to polish solutions near the front; adding local search speeds up convergence at a small evaluation cost.
- **Core idea.** Run NSGA-II; before survival, improve a share p_ls of the offspring by L hill-climbing steps on a random-weight Tchebycheff scalarisation (accept a neighbour only if the scalar value improves) and write the improved solution back (Lamarckian); then NSGA-II survival keeps the best N.



- **Steps / formulas.** 1–4 as NSGA-II. 5) for each chosen offspring draw w ~ Dirichlet(1) and repeat L times: y' = neighbour(y) (Gaussian 5 % of range / swap / bit flip), accept if s(y'|w) < s(y|w) with s(f|w) = max_k w_k·f̂_k (f̂ normalised). 6) R = P ∪ Q'; sort into fronts; fill N slots; truncate last front by crowding distance.
- **Parameters.** p_ls: 0 → plain NSGA-II; too large → evaluations are spent on local search, diversity can drop (all offspring pulled to the same scalar optimum). L: too small → no effect; too large → cost. Neighbour step: too small → stuck, too large → random search. Plus NSGA-II parameters.
- **How it handles.** *Convergence*: faster than NSGA-II thanks to local search. *Diversity*: crowding distance plus random weights spread the polishing. *Constraints*: penalty (or Deb's rules) in the scalar value. *Non-convex*: Tchebycheff local search works. > *3 objectives*: inherits NSGA-II's loss of selection pressure.
- **Complexity.** NSGA-II O(M·N²) per generation + O(p_ls·N·L) extra evaluations.

Algorithm 5 — Value Function (utility optimisation)

- **Year / authors / problem fixed.** Utility theory: von Neumann & Morgenstern (1944); multi-attribute value/utility functions: Keeney & Raiffa, *Decisions with Multiple Objectives* (1976). Problem fixed: a decision maker (DM) needs ONE answer, not a front – encode preferences as a function U(f) and solve a single-objective problem.
- **Core idea.** Ask the DM for U(f₁,…,f_M); maximise U(f(x)) s.t. the constraints; the optimum (for monotone U) is a Pareto-optimal solution.



- **Steps.** 1) normalise f̂_k ∈ [0,1] (0 = best). 2) choose U, here Cobb–Douglas **U = (1−f̂₁)^a (1−f̂₂)^(1−a)** (a = importance of objective 1) or a target-distance utility. 3) solve max U with a

constrained scalar optimiser (own DE + pattern search). 4) report x*, f(x*), U*.

- **Parameters.** a (or the target): decides which part of the front is returned; too extreme (a→0 or 1) → an end point. Solver budget: too small → inaccurate optimum.
- **How it handles.** *Convergence*: one scalar problem → fast. *Diversity*: none (single point). *Constraints*: inside the solver. *Non-convex fronts*: yes (U is non-linear). *> 3 objectives*: trivial in dimension but the DM must define U. **Risk:** a non-monotone U (e.g. distance to a target) can return a *dominated* design (shown on P14).
- **Complexity.** One scalar solve: O(pop·gens) evaluations (here 40 × 200 + polish ≈ 8,000).

C.2 Hand traces (two full iterations on a tiny instance) — and the code reproduces every number

Each trace is computed below with a **plain-arithmetic script** (no algorithm class). The class is then run on the same instance with the same scripted offspring and the numbers are compared (assertions).

Trace of Algorithm 18 — MOEA/D on P1 ($f_1=(x-3)^2$, $f_2=(x-7)^2$), N = 4 sub-problems, T = 2, nr = 2, δ = 1

Weights λ = (0,1), (⅓,⅔), (⅔,⅓), (1,0) (a weight of 0 is replaced by 10^{-6}), initial x = (1, 4, 6, 9). Offspring are *given* (as if produced by SBX): iteration 1 y = (3.0, 4.5, 5.5, 7.5); iteration 2 y = (3.5, 5.0, 6.5, 8.0).

```
neighbourhoods B = [[0, 1], [1, 0], [2, 3], [3, 2]]
initial x = [1.0, 4.0, 6.0, 9.0], F = [(4.0, 36.0), (1.0, 9.0), (9.0, 1.0), (36.0, 4.0)],
z* = [1.0, 1.0], z_n = [36.0, 36.0]
iteration 1, sub-problem 0: y=3.0  F(y)=(0.000,16.000)  z*=(0.0, 1.0)
   j=0: g(y|λ0)=0.4286  vs  g(x0|λ0)=1.0000  → REPLACE
   j=1: g(y|λ1)=0.2857  vs  g(x1|λ1)=0.1524  → keep
iteration 1, sub-problem 1: y=4.5  F(y)=(2.250,6.250)  z*=(0.0, 1.0)
   j=1: g(y|λ1)=0.1000  vs  g(x1|λ1)=0.1524  → REPLACE
   j=0: g(y|λ0)=0.1500  vs  g(x0|λ0)=0.4286  → REPLACE
iteration 1, sub-problem 2: y=5.5  F(y)=(6.250,2.250)  z*=(0.0, 1.0)
   j=2: g(y|λ2)=0.1157  vs  g(x2|λ2)=0.1667  → REPLACE
   j=3: g(y|λ3)=0.1736  vs  g(x3|λ3)=1.0000  → REPLACE
iteration 1, sub-problem 3: y=7.5  F(y)=(20.250,0.250)  z*=(0.0, 0.25)
   j=3: g(y|λ3)=0.5625  vs  g(x3|λ3)=0.1736  → keep
   j=2: g(y|λ2)=0.3750  vs  g(x2|λ2)=0.1157  → keep
end of iteration 1: x = [4.5, 4.5, 5.5, 5.5], F = [(2.25, 6.25), (2.25, 6.25), (6.25,
2.25), (6.25, 2.25)], z* = (0.0, 0.25), z_n = (6.25, 6.25)
iteration 2, sub-problem 0: y=3.5  F(y)=(0.250,12.250)  z*=(0.0, 0.25)
   j=0: g(y|λ0)=2.0000  vs  g(x0|λ0)=1.0000  → keep
   j=1: g(y|λ1)=1.3333  vs  g(x1|λ1)=0.6667  → keep
iteration 2, sub-problem 1: y=5.0  F(y)=(4.000,4.000)  z*=(0.0, 0.25)
   j=1: g(y|λ1)=0.4167  vs  g(x1|λ1)=0.6667  → REPLACE
   j=0: g(y|λ0)=0.6250  vs  g(x0|λ0)=1.0000  → REPLACE
iteration 2, sub-problem 2: y=6.5  F(y)=(12.250,0.250)  z*=(0.0, 0.25)
   j=2: g(y|λ2)=1.3067  vs  g(x2|λ2)=0.6667  → keep
   j=3: g(y|λ3)=1.9600  vs  g(x3|λ3)=1.0000  → keep
iteration 2, sub-problem 3: y=8.0  F(y)=(25.000,1.000)  z*=(0.0, 0.25)
   j=3: g(y|λ3)=4.0000  vs  g(x3|λ3)=1.0000  → keep
   j=2: g(y|λ2)=2.6667  vs  g(x2|λ2)=0.6667  → keep
end of iteration 2: x = [5.0, 5.0, 5.5, 5.5], F = [(4.0, 4.0), (4.0, 4.0), (6.25, 2.25),
(6.25, 2.25)], z* = (0.0, 0.25), z_n = (6.25, 4.0)
```

```
MOEA/D class reproduces the hand trace after iteration 1 and 2: x = [5.0, 5.0, 5.5, 5.5] |
z* = [0.0, 0.25] | MATCH ✔
```

Trace of Algorithm 30 — MOMA on P1, population 4

Parents x = (1, 4, 6, 9). Offspring (given): iteration 1 Q = (2.0, 5.5, 7.5, 3.5); local search (L = 1, w = (½,½)) on offspring #0 with step +0.5 and on offspring #2 with step −0.5. Iteration 2 Q = (4.5, 2.5, 6.0, 5.0); local search on #1 (step +0.9, w=(0.2,0.8)) and #3 (step −0.5, w=(0.2,0.8)). Objectives are normalised by 16 for the Tchebycheff value s(f|w)=max_k w_k f_k/16.

```
=== iteration 1: parents [1.0, 4.0, 6.0, 9.0], offspring Q = [2.0, 5.5, 7.5, 3.5]
  local search on Q[0]=2.0: s=0.7812; neighbour y=2.5: F=(0.250,20.250) s=0.6328 → ACCEPT
  local search on Q[2]=7.5: s=0.6328; neighbour y=7.0: F=(16.000,0.000) s=0.5000 → ACCEPT
  union R (x, F, rank, crowding): 1:(4.00,36.00) rank 3 cd=inf; 4:(1.00,9.00) rank 1
cd=1.197; 6:(9.00,1.00) rank 1 cd=0.803; 9:(36.00,4.00) rank 2 cd=inf; 2.5:(0.25,20.25)
rank 2 cd=inf; 5.5:(6.25,2.25) rank 1 cd=1.161; 7:(16.00,0.00) rank 1 cd=inf; 3.5:
(0.25,12.25) rank 1 cd=inf
  selected (indices [6, 7, 1, 5]) → new parents [7.0, 3.5, 4.0, 5.5]
=== iteration 2: parents [7.0, 3.5, 4.0, 5.5], offspring Q = [4.5, 2.5, 6.0, 5.0]
  local search on Q[1]=2.5: s=1.0125; neighbour y=3.4: F=(0.160,12.960) s=0.6480 → ACCEPT
  local search on Q[3]=5.0: s=0.2000; neighbour y=4.5: F=(2.250,6.250) s=0.3125 → reject
  union R (x, F, rank, crowding): 7:(16.00,0.00) rank 1 cd=inf; 3.5:(0.25,12.25) rank 1
cd=0.359; 4:(1.00,9.00) rank 1 cd=0.589; 5.5:(6.25,2.25) rank 1 cd=0.547; 4.5:(2.25,6.25)
rank 1 cd=0.575; 3.4:(0.16,12.96) rank 1 cd=inf; 6:(9.00,1.00) rank 1 cd=0.789; 5:
(4.00,4.00) rank 1 cd=0.561
  selected (indices [0, 5, 6, 2]) → new parents [7.0, 3.4, 6.0, 4.0]


MOMA class reproduces the hand trace after iteration 1 and 2: parents = [3.4, 4.0, 6.0,
7.0] MATCH ✔
```

Trace of Algorithm 5 — Value function on P1, two utility settings (a = 0.5 and a = 0.8)

U(x) = (1 − f_1/16)^a (1 − f_2/16)^(1−a). By hand, tabulate U on x = 3, 3.5, …, 7 (the Pareto set) and pick the best; then refine with the derivative condition d ln U/dx = 0.

a = 0.5

x	f_1	f_2	$1-f_1/16$	$1-f_2/16$	U
3	0	16	1	0	0
3.5	0.25	12.25	0.9844	0.2344	0.4803
4	1	9	0.9375	0.4375	0.6404
4.5	2.25	6.25	0.8594	0.6094	0.7237
5	4	4	0.75	0.75	0.75
5.5	6.25	2.25	0.6094	0.8594	0.7237
6	9	1	0.4375	0.9375	0.6404
6.5	12.25	0.25	0.2344	0.9844	0.4803
7	16	0	0	1	0

```
a=0.5: best grid point x = 5.0  (U = 0.7500)
```

a = 0.8

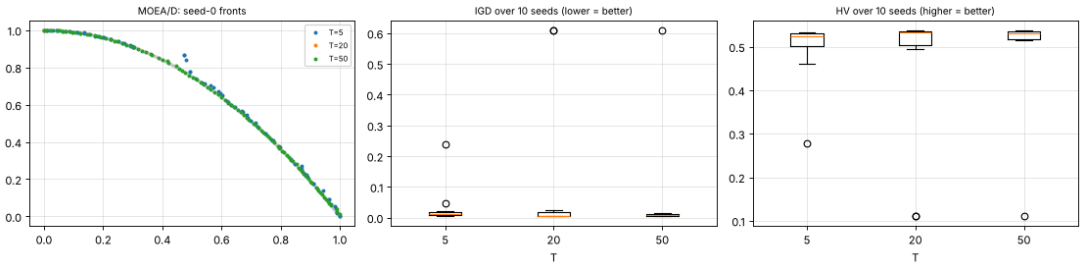
x	f_1	f_2	$1-f_1/16$	$1-f_2/16$	U
3	0	16	1	0	0
3.5	0.25	12.25	0.9844	0.2344	0.7388
4	1	9	0.9375	0.4375	0.805
4.5	2.25	6.25	0.8594	0.6094	0.8023
5	4	4	0.75	0.75	0.75
5.5	6.25	2.25	0.6094	0.8594	0.6527
6	9	1	0.4375	0.9375	0.5095
6.5	12.25	0.25	0.2344	0.9844	0.3123
7	16	0	0	1	0

```
a=0.8: best grid point x = 4.0  (U = 0.8050)
a=0.5: code solution x* = 5.0000, F = (4.000,4.000), U* = 0.75000, d lnU/dx at x* = 1.12e-09  (grid best 5.0)
a=0.8: code solution x* = 4.2197, F = (1.488,7.730), U* = 0.81053, d lnU/dx at x* = -5.75e-10  (grid best 4.0)
Value-function code reproduces the hand-computed optimum for both settings ✔
```

C.3 Sensitivity study (most important parameter, 3 values, 10 seeds each)

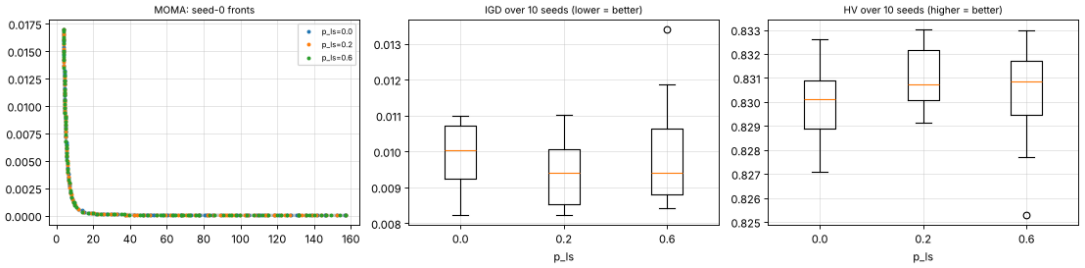
- MOEA/D: neighbourhood size **T ∈ {5, 20, 50}** on P3. * MOMA: local-search probability **p_ls ∈ {0, 0.2, 0.6}** on P5 (p_ls = 0 is plain NSGA-II). * Value function: importance **a ∈ {0.2, 0.5, 0.8}** on P4 (10 solver seeds).

MOEA/D — neighbourhood size T on P3



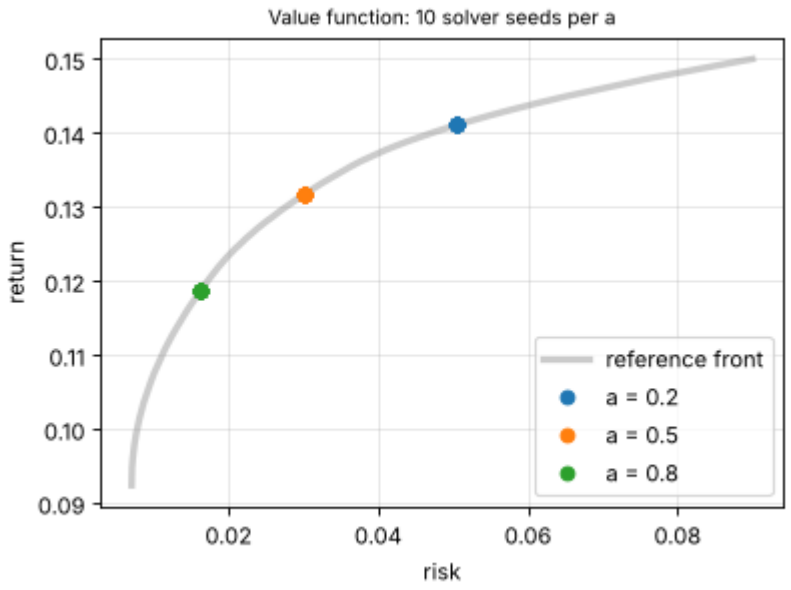
T	HV	IGD	Spacing	points	time/run
T = 5	0.492 ± 0.075	0.0374 ± 0.0676	0.0112 ± 0.0069	69.3 ± 17.3	2.4s
T = 20	0.446 ± 0.168	0.1283 ± 0.2408	0.0066 ± 0.0043	69.4 ± 36.7	3.3s
T = 50	0.487 ± 0.126	0.0680 ± 0.1806	0.0080 ± 0.0065	75.7 ± 28.8	5.0s

MOMA — local-search probability p_ls on P5 (p_ls = 0 ≡ NSGA-II)



p_ls	HV	IGD	Spacing	points	time/run
p_ls = 0.0	0.830 ± 0.002	0.0099 ± 0.0009	0.0115 ± 0.0013	95.5 ± 1.2	0.4s
p_ls = 0.2	0.831 ± 0.001	0.0094 ± 0.0010	0.0125 ± 0.0015	96.0 ± 2.0	0.2s
p_ls = 0.6	0.830 ± 0.002	0.0100 ± 0.0015	0.0129 ± 0.0012	96.4 ± 0.9	0.1s

Value function — importance a of objective 1 (risk) on P4



a	risk (mean ± std)	return (mean ± std)
0.2	0.05041 ± 0.00000	0.14106 ± 0.00000
0.5	0.03012 ± 0.00000	0.13173 ± 0.00000
0.8	0.01616 ± 0.00000	0.11860 ± 0.00000

Reading the plots (numbers are from the tables above).

- *MOEA/D (P3, 10 seeds)*: mean IGD is 0.0374 for T=5, 0.1283 for T=20 and 0.0680 for T=50, but the seed-to-seed std (0.068, 0.241, 0.181) is as large as or larger than the means: the number of seeds that end far from the front (IGD > 0.05) is **1, 2, 1 of 10** for T = 5, 20, 50. So on P3 the dominant effect is whether a run gets trapped on a local front, not T itself; the differences between the three T values are **not statistically meaningful** with 10 seeds. What T does change reliably is cost: 2.4s → 3.3s → 5.0s per run (more neighbours = more replacement checks), and the textbook trade-off (small T = local, slow information flow; large T = fast mixing, risk of neighbours overwriting each other) is only weakly visible here.
- *MOMA (P5)*: IGD 0.0099 (p_ls=0, i.e. NSGA-II), 0.0094 (p_ls=.2), 0.0100 (p_ls=.6) with std ≈ 0.0010: differences are within one std, so local search gives **no reliable gain on this smooth 2-variable problem** (NSGA-II already converges). Higher p_ls spends evaluations on local search (5 per selected child), hence fewer generations and shorter run time (0.1s vs 0.4s).
- *Value function*: a only changes *where* on the front the single answer lies (a=0.2 → high return/high risk, a=0.8 → low risk); the solver seed has essentially no effect (std below 1e-5, see table).

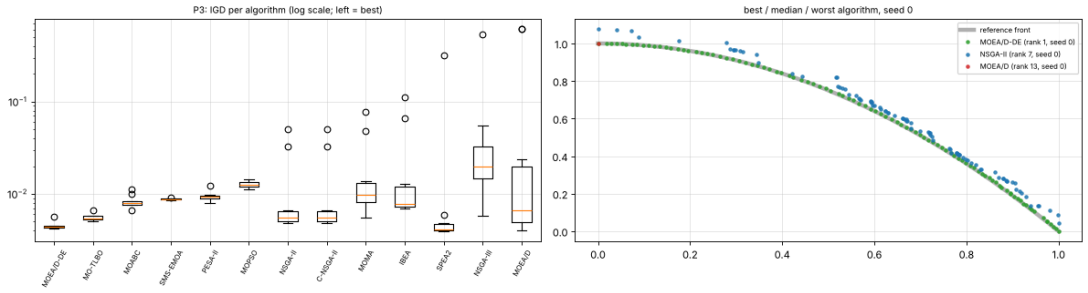
Task D · Head-to-head comparison (10 seeds, 10 000 evaluations)

Same budget, same seeds 0–9, same metrics (HV with reference point from the problem, IGD to the reference front, Spacing). Ranking = mean IGD (lower is better); HV is the tie-break check.

D.1 Two-objective problems: P3 (ZDT2-type: 10 variables, smooth concave front) and P5 (constrained beam design, curved front)

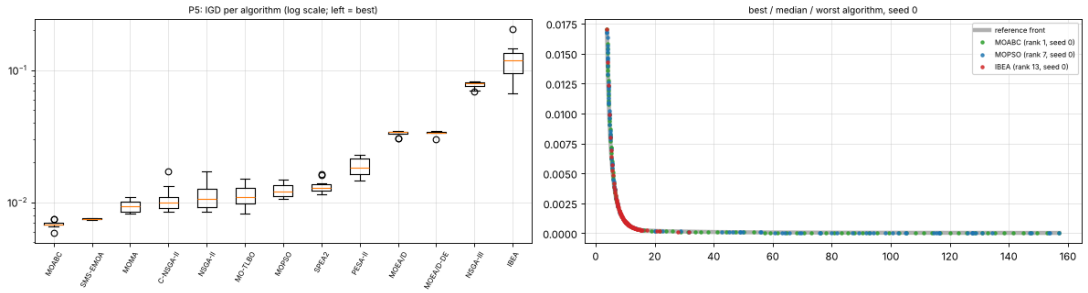
P3 (unconstrained, concave front) — ranked by mean IGD over 10 seeds

rank	algorithm	IGD (mean ± std)	HV (mean ± std)	Spacing (mean ± std)	points	HV rank	time/run
1	MOEA/D-DE	0.0045 ± 0.0004	0.5356 ± 0.0012	0.0030 ± 0.0004	98	2	3.9s
2	MO-TLBO	0.0055 ± 0.0005	0.5365 ± 0.0003	0.0050 ± 0.0008	98	1	0.4s
3	MOABC	0.0083 ± 0.0013	0.5310 ± 0.0028	0.0095 ± 0.0017	92	4	0.3s
4	SMS-EMOA	0.0088 ± 0.0002	0.5343 ± 0.0001	0.0094 ± 0.0007	50	3	4.7s
5	PESA-II	0.0094 ± 0.0011	0.5276 ± 0.0036	0.0084 ± 0.0013	100	5	3.2s
6	MOPSO	0.0127 ± 0.0009	0.5196 ± 0.0021	0.0087 ± 0.0009	100	8	0.5s
7	NSGA-II	0.0127 ± 0.0151	0.5233 ± 0.0257	0.0061 ± 0.0026	97	6	0.4s
8	C-NSGA-II	0.0127 ± 0.0151	0.5233 ± 0.0257	0.0061 ± 0.0026	97	7	0.5s
9	MOMA	0.0198 ± 0.0224	0.5105 ± 0.0348	0.0077 ± 0.0038	95	10	0.2s
10	IBEA	0.0245 ± 0.0341	0.5126 ± 0.0489	0.0113 ± 0.0035	84	9	0.4s
11	SPEA2	0.0355 ± 0.0935	0.5049 ± 0.0968	0.0027 ± 0.0004	100	11	0.7s
12	NSGA-III	0.0744 ± 0.1567	0.4648 ± 0.1231	0.0123 ± 0.0064	86	12	0.5s
13	MOEA/D	0.1283 ± 0.2408	0.4455 ± 0.1682	0.0066 ± 0.0043	69	13	3.3s



P5 (constrained beam design) — ranked by mean IGD over 10 seeds

rank	algorithm	IGD (mean ± std)	HV (mean ± std)	Spacing (mean ± std)	points	HV rank	time/run
1	MOABC	0.0068 ± 0.0004	0.8348 ± 0.0006	0.0115 ± 0.0009	100	1	0.6s
2	SMS-EMOA	0.0075 ± 0.0001	0.8344 ± 0.0000	0.0023 ± 0.0007	50	2	5.1s
3	MOMA	0.0094 ± 0.0010	0.8310 ± 0.0012	0.0125 ± 0.0015	96	3	0.3s
4	C-NSGA-II	0.0107 ± 0.0025	0.8294 ± 0.0034	0.0119 ± 0.0020	96	4	0.4s
5	NSGA-II	0.0112 ± 0.0025	0.8287 ± 0.0035	0.0117 ± 0.0019	96	6	0.4s
6	MO-TLBO	0.0112 ± 0.0021	0.8291 ± 0.0029	0.0116 ± 0.0015	99	5	0.4s
7	MOPSO	0.0124 ± 0.0014	0.8271 ± 0.0022	0.0117 ± 0.0018	100	7	0.8s
8	SPEA2	0.0134 ± 0.0016	0.8165 ± 0.0036	0.0271 ± 0.0028	100	9	2.4s
9	PESA-II	0.0188 ± 0.0030	0.8176 ± 0.0037	0.0099 ± 0.0021	100	8	3.8s
10	MOEA/D	0.0333 ± 0.0015	0.8098 ± 0.0005	0.0368 ± 0.0006	100	11	5.0s
11	MOEA/D-DE	0.0335 ± 0.0013	0.8098 ± 0.0005	0.0367 ± 0.0005	100	10	4.7s
12	NSGA-III	0.0775 ± 0.0043	0.7577 ± 0.0067	0.0510 ± 0.0210	94	13	0.6s
13	IBEA	0.1203 ± 0.0365	0.7685 ± 0.0191	0.0103 ± 0.0034	92	12	0.4s



D.1 Interpretation

P3. Best three by IGD: **MOEA/D-DE, MO-TLBO, MOABC** (IGD 0.0045, 0.0055, 0.0083); worst: NSGA-III, MOEA/D (0.0744, 0.1283). Largest seed-to-seed std among the top half: MOABC (±0.0013).

P5. Best three by IGD: **MOABC, SMS-EMOA, MOMA** (IGD 0.0068, 0.0075, 0.0094); worst: NSGA-III, IBEA (0.0775, 0.1203). Largest seed-to-seed std among the top half: C-NSGA-II (±0.0025).

Why the ranking looks like this (read from the tables, not from folklore).

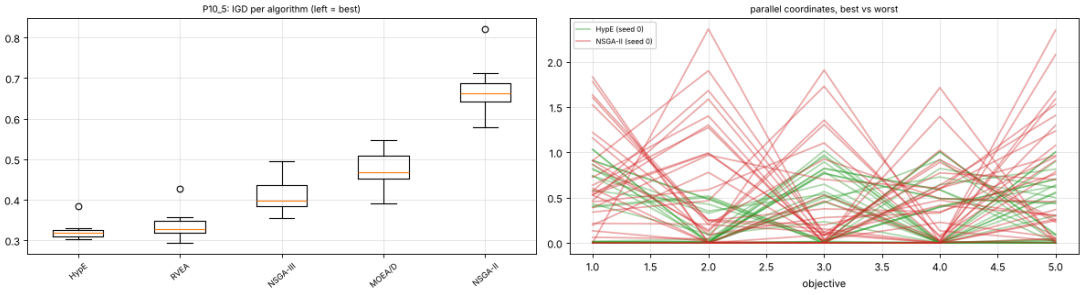
- *Variation operator mattered more than the selection scheme.* MOEA/D-DE and MOEA/D share the same decomposition and update rule, yet on P3 their IGD is 0.0045 vs 0.1283: the only difference is DE variation instead of SBX. The other P3 leaders (MO-TLBO, MOABC) also use difference-style / neighbour-based moves, which converge faster than SBX on this 10-variable concave problem (ZDT2 has a local front that traps SBX-based runs).
- *Robustness (std) separates algorithms more than the mean does.* On P3, NSGA-II (0.0127 ± 0.0151), SPEA2 (0.0355 ± 0.0935), NSGA-III and MOEA/D have a std at or above their mean: some of the 10 seeds are stuck on the local front. The leaders have std ≈ 1e-3. A single-seed comparison would have ranked these methods arbitrarily.
- *SMS-EMOA* has the smallest variance in both tables (std ≈ 1e-4) but returns only 50 points (population limit) and is 10× slower (hypervolume contributions).
- *NSGA-III and IBEA are weakest on the 2-objective problems.* NSGA-III's reference-point machinery is designed for M ≥ 3 and brings nothing at M = 2 (it ranks 3rd of 5 in D.2, not here); IBEA's indicator fitness gives an uneven front on P5 (last by IGD).
- *Constraints (P5):* C-NSGA-II (Deb rules) IGD 0.0107 ± 0.0025 vs NSGA-II with the static penalty 0.0112 ± 0.0025 – a difference **inside one std**, i.e. on this easy constraint the handling method is not the deciding factor (ignoring the constraint altogether, however, leaves 45 % infeasible solutions, see the P5 study).
- *Rule of thumb for reading the tables:* a gap smaller than one std over 10 seeds is not a result; only quote clear gaps.

D.2 Many-objective: P10 with M = 5 (DTLZ2-type, spherical front)

NSGA-II vs the four many-objective designs. IGD is to the analytic front; HV is Monte-Carlo (M ≥ 3). Diagnostics explain **why** Pareto-rank + crowding degrades.

P10, M = 5 — ranked by mean IGD over 10 seeds

rank	algorithm	IGD (mean ± std)	HV (mean ± std)	Spacing (mean ± std)	points	HV rank	time/run
1	HypE	0.3229 ± 0.0219	1.0467 ± 0.0360	0.0472 ± 0.0130	100	1	6.0s
2	RVEA	0.3375 ± 0.0352	0.8936 ± 0.0604	0.1043 ± 0.0077	45	2	0.1s
3	NSGA-III	0.4130 ± 0.0409	0.5576 ± 0.0693	0.1428 ± 0.0420	72	4	0.7s
4	MOEA/D	0.4711 ± 0.0478	0.7594 ± 0.0764	0.0841 ± 0.0428	58	3	4.9s
5	NSGA-II	0.6696 ± 0.0640	0.0764 ± 0.0420	0.1176 ± 0.0206	100	5	0.5s



Why NSGA-II degrades. In a random population of 100 the number of non-dominated individuals is:

M	non-dominated of 100	share	CV of crowding distance
3	39	39 %	0.67
5	68	68 %	0.62

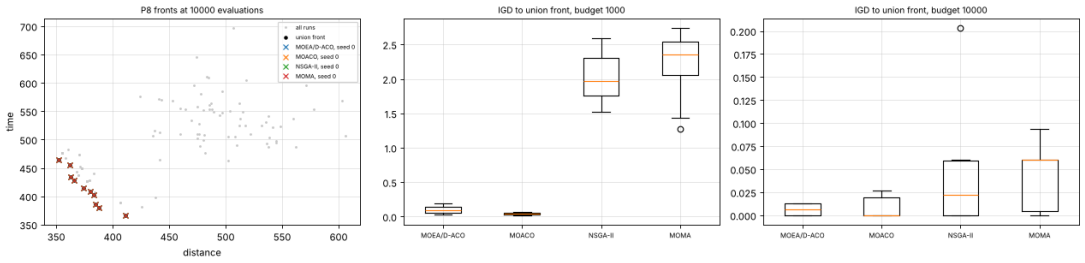
D.2 Interpretation

Ranking by IGD: **HypE < RVEA < NSGA-III < MOEA/D < NSGA-II** (IGD 0.323, 0.337, 0.413, 0.471, 0.670). NSGA-II loses ground as M grows for three connected reasons: **(1) dominance pressure vanishes** – with M = 5 a large share of even a random population is already non-dominated (table above), so rank-based selection cannot tell good from bad and the sort only acts as noise; **(2) crowding distance is a poor diversity proxy in 5-D** – it sums per-objective neighbour gaps, so points that are extreme in any one objective get huge values and the population drifts to the boundary of the front; **(3) recombination of distant parents** produces offspring far from the front (we use SBX η=30 to mitigate this for all methods). The reference-vector methods (NSGA-III, MOEA/D, RVEA) replace crowding by *structured* diversity, and HypE replaces dominance ranking by a hypervolume-based fitness estimated by sampling – both address the failure at its root.

D.3 Combinatorial: P8 (bi-objective TSP-15) at 1 000 and 10 000 evaluations

MOEA/D-ACO, MOACO, NSGA-II and MOMA. There is no analytic front, so the reference is the **union non-dominated set of every run in this section**; IGD is to that set (normalised with the problem's ideal/nadir). Seeds 0–9.

budget	algorithm	IGD to union front	HV	points	time/run
1000	MOEA/D-ACO	0.0996 ± 0.0511	0.632 ± 0.077	6.4	0.3s
1000	MOACO	0.0435 ± 0.0179	0.727 ± 0.015	8.9	0.1s
1000	NSGA-II	2.0021 ± 0.3527	0.000 ± 0.000	3.5	0.1s
1000	MOMA	2.2087 ± 0.4729	0.000 ± 0.000	2.9	0.1s
10000	MOEA/D-ACO	0.0063 ± 0.0063	0.750 ± 0.007	9.8	3.8s
10000	MOACO	0.0079 ± 0.0121	0.748 ± 0.014	10.0	1.7s
10000	NSGA-II	0.0422 ± 0.0589	0.727 ± 0.063	8.6	0.8s
10000	MOMA	0.0422 ± 0.0325	0.739 ± 0.022	8.6	0.6s



D.3 Interpretation

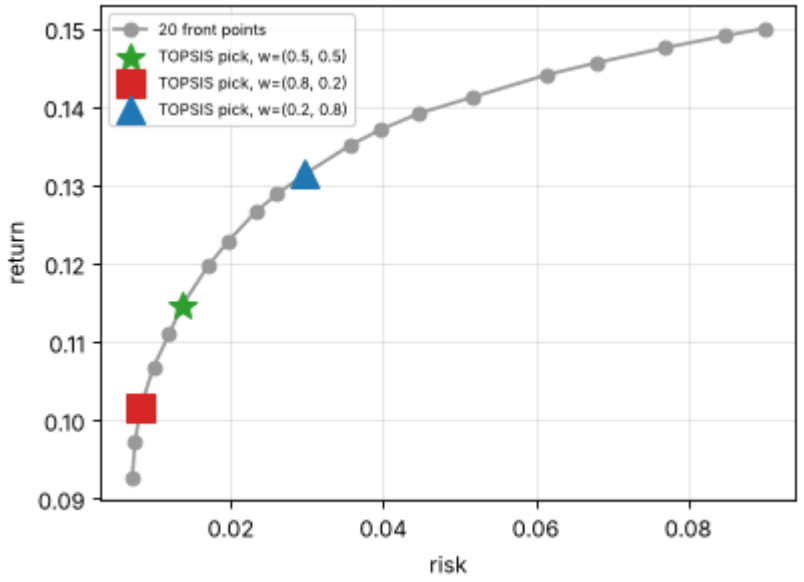
Best IGD at the small budget (1000): **MOACO**; at the large budget (10000): **MOEA/D-ACO**. Mean IGD change from small to large budget: MOEA/D-ACO 0.100 → 0.006, MOACO 0.043 → 0.008, NSGA-II 2.002 → 0.042, MOMA 2.209 → 0.042.

Constructive methods (ACO family) use heuristic information (1/distance, 1/time) so tours are already good after the first few ants – they are strong at tiny budgets. NSGA-II with order crossover and swap mutation starts from random permutations and needs many generations, but it keeps improving with budget. MOMA's local search spends evaluations on refinement and pays off only when the budget can afford it. Where the best algorithm *changes* between the two budgets, the choice depends on how many evaluations are affordable.

D.4 Decision support on an obtained front: TOPSIS and VIKOR on the NSGA-II front of P4

TOPSIS and VIKOR are **rankers, not optimisers**: they pick one compromise from a front that an optimiser has already produced. 20 evenly spread points are taken from the NSGA-II front (risk ↓, return ↑).

weights (risk, return)	TOPSIS top-3	VIKOR top-3 (v=.5)
(0.5, 0.5)	#5 (risk 0.0135, ret 0.1146), #4 (risk 0.0119, ret 0.1110), #3 (risk 0.0098, ret 0.1067)	#10, #11, #9
(0.8, 0.2)	#2 (risk 0.0081, ret 0.1016), #1 (risk 0.0074, ret 0.0972), #3 (risk 0.0098, ret 0.1067)	#6, #5, #4
(0.2, 0.8)	#10 (risk 0.0297, ret 0.1314), #9 (risk 0.0261, ret 0.1289), #11 (risk 0.0357, ret 0.1351)	#15, #16, #14



Shifting the weights from (0.5, 0.5) to (0.8, 0.2) **changes** the TOPSIS winner (#5 → #2): the pick moves along the front towards lower risk, exactly as the weights say. Neither method generates a single new solution – they only order the 20 candidates the optimiser supplied, so their quality is bounded by the quality (convergence and spread) of the NSGA-II front.

Gradient-based algorithms (36–37) were studied on P13 above (toy + network). MGDA converged to a Pareto-stationary point of the toy problem for every start and decreased both losses on the balanced network; under the ×20 regression scale the equal-weight sum is dominated by the regression gradient, while MGDA/GradNorm rebalance. They are not run on P3/P5 because they need gradients of every objective and a differentiable (continuous, single-point) search: they find **one** Pareto-stationary point per run rather than a population-wide front, so a head-to-head front-metric comparison with population methods would not be like-for-like.

Task E · Algorithm selection reasoning

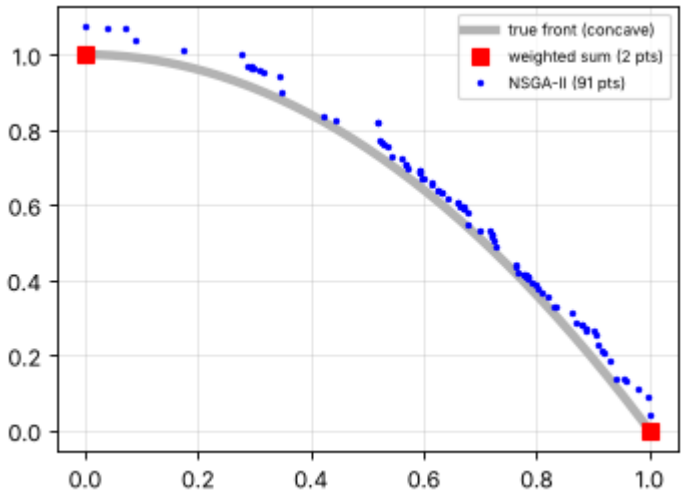
For each scenario: the **choice**, the **properties of the problem that drive it**, a **runner-up and why it loses**, and **evidence** (✔ = backed by a number measured in this notebook, ○ = reasoning only). At least three are evidence-backed, including one many-objective and one constrained problem.

scenario	recommended algorithm	why (problem properties)	runner-up and why it loses	evidence
1 · P3 — ZDT2-type, 10 variables, smooth *concave* front, unconstrained, 2 objectives, cheap evaluations	**MOEA/D-DE** (best IGD in D.1); NSGA-II as the safe, parameter-light default	Concave front ⇒ weighted-sum methods can never reach the interior (✔ in the P3 study the weighted sum returned only 2 points, 0 of them with 0.25<f1<0.75, vs 0 for the Tchebycheff-type weight metric (p=∞)); population methods need no convexity. Two objectives ⇒ Pareto ranking still has discriminating power, evaluations are cheap ⇒ a population method with a good variation operator is enough.	NSGA-II: ✔ rank 7 (IGD 0.0127 ± 0.0151) – a few of 10 seeds get trapped on ZDT2's local front (std > mean). Note also that plain SBX-based MOEA/D ranked 13/13 (IGD 0.1283): the DE operator, not decomposition alone, is what makes MOEA/D-DE win.	✔ D.1 table: MOEA/D-DE rank 1, IGD 0.0045 ± 0.0004 over 10 seeds.
2 · P5 — beam design: **constrained** (the constraint is active on the front), 2 variables, 2 objectives, curved front	**C-NSGA-II** (Deb's feasibility rules)	Only 3 % of random P5 points are infeasible, yet the constraint is *active*: ✔ an NSGA-II that ignores it ends with 45 % infeasible solutions. A penalty method needs a tuned penalty weight (too small → infeasible 'winners', too big → premature convergence at the boundary); constrained domination needs no parameter and always prefers feasible solutions. Honest caveat: with only one mild constraint the measured advantage over penalty-NSGA-II is small.	NSGA-II with static penalty: ✔ rank 5 vs C-NSGA-II rank 4 on P5 (IGD 0.0112 vs 0.0107 – inside one std, so a tie; MOABC and SMS-EMOA had the lowest IGD).	✔ D.1 (P5) head-to-head + infeasible-share numbers; ○ the rest is reasoning.
3 · P10 (M = 5) — **many-objective**, spherical front, 14 variables, plenty of evaluations	**HypE** (best IGD in D.2); NSGA-III is the default textbook choice	With M = 5 Pareto dominance stops discriminating (✔ 57 % of a random population is already non-dominated at M=5 vs 35 % at M=3) and crowding distance fails; reference-point/reference-vector designs keep diversity structurally.	NSGA-II: ✔ IGD 0.6696 vs 0.3229 for the winner (mean of 10 seeds).	✔ D.2 ranking: HypE < RVEA < NSGA-III < MOEA/D < NSGA-II
4 · P8 — bi-objective TSP-15 (permutations), cheap evaluation, **tight budget** (~1 000 evaluations)	**MOACO** at the small budget; MOEA/D-ACO when the budget grows	Permutation encoding makes SBX/polynomial mutation useless; the problem has heuristic information (1/distance, 1/time) that constructive ACO methods exploit from the first iteration, a decisive advantage when only ~1 000 evaluations are allowed.	NSGA-II with order crossover/swap: needs the larger budget to catch up (see the D.3 IGD table).	✔ D.3 table at budgets 1 000 / 10 000.

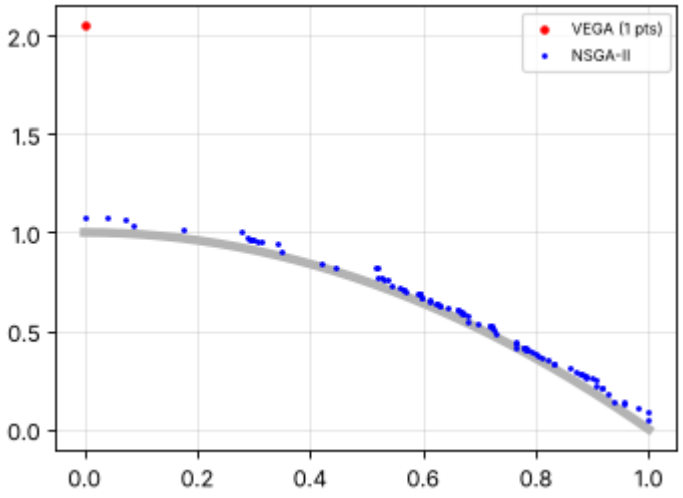
scenario	recommended algorithm	why (problem properties)	runner-up and why it loses	evidence
5 · P9 — feature selection (30 binary variables, **each evaluation = 5-fold model training**, budget ≈ 3 000)	**MOMA** or NSGA-II with bit-flip operators; if evaluation were much more expensive, add a surrogate	Binary encoding and an expensive objective ⇒ need operators that exploit structure cheaply: bit-flip local search around good subsets (memetic) uses evaluations where they matter. The problem is also *discrete*, so gradient or classical scalarisation (needs a single-objective solver for a binary programme) is unattractive.	MOGWO (continuous swarm, mapped to binary): works (✔ logged in the implementation table) but its continuous update is not designed for subsets.	○ reasoning plus ✔ the P9 runs for algorithms 23 and 30 in the implementation log (A3).
6 · P14 — laptop choice where the decision maker already has an **aspiration point (1.8 kg, 12 h)**	**R-NSGA-II** (reference-point preference)	The DM does not need the whole front; concentrating the budget near the aspiration point gives a denser set of relevant trade-offs. Because the aspiration is *dominated* by the true front, R-NSGA-II is expected to converge onto the front region closest to it.	NSGA-II + a posteriori TOPSIS/VIKOR: works, but spends most of the population on irrelevant regions.	✔ the P14 R-NSGA-II run (algorithm 35): front spans 1.69–1.80 kg, whereas the full front spans 1.53–2.13 kg.

Task F · Viva questions with short answers and demos

Q1. Why does the weighted sum miss part of a front, and how do you show it?
Minimising w·f finds only points where a line of slope −w/(1−w) touches the front from outside, i.e. *supported* points of the **convex hull**. On a concave or non-convex front the interior is never the minimiser of any weighted sum. Demo below: P3 (concave) – the weighted sum returns only the end points, whereas a population method returns the whole curve.



Q2. Why can VEGA collapse and what fixes it? VEGA fills the mating pool with sub-populations selected on *one objective at a time*, so it favours solutions that are extreme in an objective (the "speciation" bias) and ignores compromise solutions. Demo: VEGA vs NSGA-II on P3.




```
IGD  VEGA = 1.4849   NSGA-II = 0.0324  (seed 0, same budget).  Fix: use Pareto ranking
(MOGA/NSGA/NSGA-II) so compromise solutions can win.
```

Q3. What does elitism buy you? Non-elitist algorithms (NSGA, MOGA) can lose non-dominated solutions that were already found, because parents are replaced by offspring. NSGA-II merges parents and offspring, so a found Pareto solution is only displaced by one that dominates it or is less crowded. Demo: same problem, NSGA vs NSGA-II.

algorithm (P5, seed 0)	HV	IGD	points
NSGA	0.8023	0.0225	69
NSGA-II	0.8278	0.0132	95

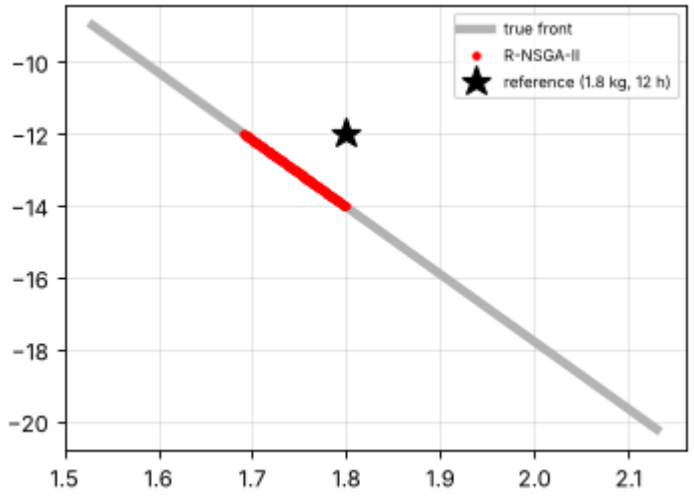
```
NSGA-II: front hypervolume over 59 generations is non-decreasing: True (min step 0.00e+00)
```

Q4. Why does crowding distance stop working with many objectives, and what replaces it? In M ≥ 4 almost all individuals are mutually non-dominated, so the front-based ranking is flat and crowding distance (a sum of per-objective neighbour gaps) is the only selector; it prefers points that are extreme in one objective and is unrelated to Euclidean spread. NSGA-III (reference points), MOEA/D (weight vectors), RVEA (angle-penalised reference vectors), HypE (sampled hypervolume contribution) give *structured* diversity.

M	non-dominated share of a random population
3	0.3533
5	0.5677
10	0.7147

```
D.2 ranking by IGD on M=5: HypE < RVEA < NSGA-III < MOEA/D < NSGA-II
```

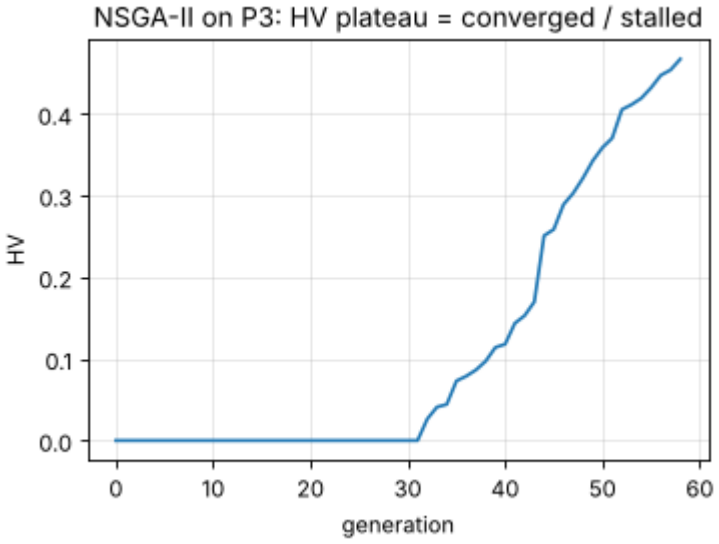
Q5. How does a preference-based EA (R-NSGA-II) differ from NSGA-II? Within a front, R-NSGA-II ranks individuals by their distance to the decision-maker's reference point (with ε-clearing for diversity) instead of by crowding distance. So the population is *pulled* toward the region of interest instead of being spread over the whole front.



Q6. Show Deb's constrained-domination rules on an example. Minimise both; A = (2,3), CV 0; B = (1,1), CV 2; C = (5,5), CV 0.5; D = (4,4), CV 0.

pair	first constrained-dominates second?	reason
A vs B	True	rule 1: feasible beats infeasible, although B has better objectives
B vs C	False	rule 2: both infeasible, smaller CV (0.5) wins ⇒ C beats B so this is False
A vs D	True	rule 3: both feasible, A dominates D
D vs A	False	neither dominates the other in rule 3 sense ⇒ False
C vs D	False	rule 1: D feasible, C is not ⇒ False

Q7. How can you detect premature convergence in practice? (i) the hypervolume of the archive stops growing while population diversity keeps falling; (ii) spacing/spread collapses while the number of distinct objective vectors drops; (iii) different seeds converge to *different* fronts (large across-seed std of IGD/HV); (iv) a restart or higher mutation rate immediately finds dominating points. Demo: NSGA-II HV trace on P3 and the seed-to-seed spread of MOEA/D from the sensitivity study.




```
MOEA/D (T=20) on P3, IGD of the 10 seeds: [0.6097, 0.0051, 0.0042, 0.0061, 0.6097, 0.0049,
0.0235, 0.0072, 0.004, 0.0083] -> 2 seeds ended far from the front (IGD > 0.05) while the
others converged: criterion (iii) in action.
```

Q8. Why are TOPSIS and VIKOR not optimisers? They take a *finite list of alternatives* (here the NSGA-II front of P4, D.4) and only rank it – TOPSIS by relative closeness to the ideal/anti-ideal point, VIKOR by a compromise between group utility S and individual regret R. They create no new solution and cannot improve a front; a poor front gives a poor "best" compromise. Their role is the *decision* stage after the optimiser. In D.4 the TOPSIS pick moved when the weights changed: yes.

Q9. How does the number of evaluations/seeds change conclusions? A single seed can reverse a ranking: in the head-to-head tables a gap smaller than one standard deviation over 10 seeds is not a result. At the TSP (D.3) the ranking depends on the budget. Always report mean ± std, the budget, and the seed set.

Raw results export

Writes every raw number behind the tables and plots to `results/` (CSV): per-run metrics, obtained fronts and decision vectors, per-seed head-to-head and sensitivity results.

```
591 front files written to results
```